Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Departamento de Ingeniería Eléctrica

Tarea N°1

Clasificador de Bayes

Integrantes: Diego Vega
Profesor: Javier Ruiz del Solar
Auxiliar: Patricio Loncomilla
Ayudantes: Benjamín Llancao Mella
Bruno Cavieres L.
Gonzalo Olguín M.

Fecha de realización: 15 de abril de 2024
Fecha de entrega: 14 de abril de 2024
Santiago de Chile

Introducción

En el presente informe se detalla el desarrollo de la tarea nº1 del curso Inteligencia Computacional. El desarrollo de la tarea se hizo acorde a las indicaciones del documento *Tarea_1_EL4106_2024_v02.pdf*.
En el documento mencionado anteriormente tiene como objetivo el estudio de algunos clasificadores en base a Bayes. De manera complementaria, se realizarán instancias de análisis y visualización de datos. El conjunto de datos utilizado durante la tarea sera *MAGIC Gamma Telescope Data Set*, el cual fue entregado por el cuerpo docente en el archivo *magic04.data*. El Data Set proviene de simulaciones de air showers generados por rayos gamma primarios v/s hadrones. El Data Set posee 11 columnas, las primeras 10 corresponden a las características o **features**, mientras que la ultima corresponde a la clase o **class** del objeto. Existen dos clases **Gamma** y **Hadron**.
Los objetivos de la tarea, y por tanto lo que se ha realizado, consisten en:

Visualizar y analizar datos Esta sección consiste en: La visualización de los histogramas de características de cada clase, calculo y gráfico de la matriz de correlación de características, obtener las características más correlacionadas con la clase, obtener las primeras tres y las ultimas tres características más correlacionadas entre si y realizar un gráfico de dispersión de las dos características más y menos correlacionadas entre sí.

Implementación de un clasificador utilizando Naive Bayes e histogramas Esta sección consiste en: Separar el Data Set en conjuntos de entrenamiento (80 %) y prueba (20 %) (procurando mantener la proporción de las clases en dichos conjuntos), la implementación de un clasificador Naive Bayes que use histogramas para representar las verosimilitudes, definición de los Bins a utilizar para todos los casos, obtención de los histogramas de las clases y los gráficos de estos mismos (para todas las características y en un mismo gráfico), prueba del clasificador y obtención de la curva ROC y la curva Precision-Recall para distintos valores de Θ.

Implementación de un clasificador utilizando Multivariate Gaussian Esta sección consiste en: Entrenar un modelo Multivariate Gaussian para cada clase (mediante el uso de la media y varianza), implementar el clasificador, prueba del clasificador y obtención de la curva ROC y la curva Precision-Recall para distintos valores de Θ.

Comparación de resultados de los clasificadores implementados Esta sección consiste en: Comparación de las curvas ROC y de las curvas Precision-Recall de ambos clasificadores (Se realizan ambas en un mismo gráfico) y se compara el tiempo requerido para entrenar ambos clasificadores.

Índice de Contenidos

1. Desarrollo **1**
1.1. Visualización y análisis de datos 1
1.1.1. Carga y lectura del Data Set 1
1.1.2. Separación en conjuntos según la clase 3
1.1.3. Generación de histogramas 5
1.1.4. Calculo y representación de la matriz de correlación 10
1.1.5. Orden ascendente de características relacionadas con la clase 11
1.1.6. Primeras 3 características más correlacionadas entre sí 12
1.1.7. Gráfico de las dos características más correlacionadas 14
1.1.8. Ultimas 3 características más correlacionadas entre sí 15
1.1.9. Gráfico de las dos características menos correlacionadas 16
1.2. Clasificación usando Naive Bayes e histogramas 17
1.2.1. Dividir Data Set en conjuntos de entrenamiento y prueba 17
1.2.2. Implementación de un clasificador Naive Bayes con histogramas por verosimilitudes 18
1.2.3. Calculo de histogramas para cada clase 20
1.2.4. Generación de histogramas 21
1.2.5. Curva ROC 23
1.2.6. Curva Precision-Recall 25
1.3. Clasificación usando Gaussianas Multivariantes 28
1.3.1. Entrenar un Modelo Gaussiano multidimensional para cada clase . 28
1.3.2. Implementar un clasificador Bayes con gaussianas multivariantes . . 31
1.3.3. Curva ROC 32
1.3.4. Curva Precision-Recall 34
1.4. Comparación de resultados 36
1.4.1. Comparativa de curvas ROC 36
1.4.2. Comparativa de curvas Precision-Recall 37
1.4.3. Comparativa de tiempos de entrenamiento 39

2. Conclusiones **40**
2.1. Aprendizajes 40
2.2. Dificultades 42
2.3. Evaluación de resultados 42

Anexo A. Código **42**

Índice de Figuras

1. Histogramas normalizados de las características con ambas clases separadas. . 7

2. Matriz de correlación de las características. 11
3. Gráfico de dispersión de las dos características más correlacionadas entre sí. . 14
4. Gráfico de dispersión de las dos características menos correlacionadas entre sí. 16
5. Histogramas normalizados de las características con ambas clases separadas. . 22
6. Curva ROC Naive Bayes. 25
7. Curva Precision-Recall Naive Bayes. 27
8. Curva ROC Gaussianas Multivariantes. 33
9. Curva Precision-Recall Gaussianas Multivariantes. 35
10. Gráfico comparativo de las curvas ROC de los clasificadores. 37
11. Gráfico comparativo de las curvas Precision-Recall de los clasificadores. 39

Índice de Tablas

1. Descripción de las características del Data Set. 1

Índice de Códigos

1. Carga Data Set. 1
2. Normalización de los datos. 2
3. Creación de Data Frames por clase. 3
4. Obtención de histogramas para cada clase. 5
5. Calculo y gráfica de la Matriz de correlación. 10
6. Orden ascendente de características relacionadas con la clase. 11
7. Primeras 3 características más correlacionadas entre sí 12
8. Gráfico de dispersión de las dos características más correlacionadas. 14
9. Ultimas 3 características más correlacionadas entre sí. 15
10. Gráfico de dispersión de las dos características menos correlacionadas. 16
11. Separación del Data Set en 2 conjuntos representativos (Entrenamiento y Prueba). 17
12. Implementación de un clasificador Bayesiano Ingenuo con histogramas por verosimilitudes. 19
13. Función que permite obtener los histogramas normalizados de cada clase. . . . 20
14. Obtención de histogramas para cada clase 2. 21
15. Generación de la curva ROC Bayes Ingenuo. 23
16. Generación de la curva Precision-Recall Naive Bayes. 26
17. Entrenador Gaussiano Multivariantes. 28
18. Implementación del clasificador Bayes en base a Gaussiana Multivariante. . . 31
19. Generación de la curva ROC Gaussianas Multivariantes. 32
20. Generación de la curva Precision-Recall Gaussianas Multivariantes. 34
21. Generación del gráfico comparativo de las curvas ROC. 36
22. Generación del gráfico comparativo de las curvas Precision-Recall. 37
23. Tiempo de entrenamiento de ambos clasificadores. 39

A.1. Jupyter Notebook. 42

1. Desarrollo

1.1. Visualización y análisis de datos

1.1.1. Carga y lectura del Data Set

En la presente subsección se realiza la carga del Data Set, para lo que se utiliza la librería pandas. Además, el Data Set contenido en el archivo *magic04.data* no contiene los nombres de las columnas (que representan las características de las muestras), por lo que fueron agregados de manera manual. Las columnas agregadas fueron ['fLength', 'fWidth', 'fSize', 'fConc', 'fConc1', 'fAsym', 'fM3Long', 'fM3Trans', 'fAlpha', 'fDist', 'class']. A continuación se explicaran las características de las muestras del Data Set.

Tabla 1: Descripción de las características del Data Set.

Característica	Dominio	Descripción
fLength	Continuo $\mathbb{R}^+$	Eje mayor de la elipse [mm].
fWidth	Continuo $\mathbb{R}^+$	Eje menor de la elipse [mm].
fSize	Continuo $\mathbb{R}^+$	Logaritmo de la suma del contenido de todos los píxeles [en fotones].
fConc	Continuo $\mathbb{R}+$	Cociente de la suma de los dos píxeles más altos sobre fSize [ratio].
fConc1	Continuo $\mathbb{R}^+$	Proporción del píxel más alto sobre fSize [ratio].
fAsym	Continuo $\mathbb{R}$	Distancia desde el pixel más alto hacia el centro, proyectado en el eje mayor [mm].
fM3Long	Continuo $\mathbb{R}$	Raíz cúbica del tercer momento a lo largo del eje mayor [mm].
fM3Trans	Continuo $\mathbb{R}$	Raíz cúbica del tercer momento a lo largo del eje menor [mm].
fAlpha	Continuo $\mathbb{R}^+$	Ángulo del eje mayor con el vector hacia el origen [grados].
fDist	Continuo $\mathbb{R}^+$	Distancia desde el origen al centro de la elipse [mm].
class	Discreto $[g, h]$	Gamma (g) para las señales y para los fondos Hadrón (h).

En la tabla 1 se hace mención de una elipse. Esta elipse es la representación visual de una lluvia de partículas después de un proceso de preprocesamiento.
Es relevante mencionar que el Data Set esta compuesto por 19020 muestras, de las cuales 12332 son de la clase g y 6688 de la clase h.
El código utilizado para la carga y visualización del Data Set se presenta en el siguiente bloque.

Código 1: Carga Data Set.

```
data = 'magic04.data'
column_names = ['fLength', 'fWidth', 'fSize', 'fConc', 'fConc1', 'fAsym', 'fM3Long', '
    ↪ fM3Trans', 'fAlpha', 'fDist', 'class']
magic04_df = pd.read_csv(data, names=column_names)
print("DataFrame de magic04.data con los nombres de las columnas agregados:")
print(magic04_df)
```

Lo obtenido mediante la ejecución del bloque anterior se presenta a continuación.

```
DataFrame de magic04.data con los nombres de las columnas agregados:
```

```
       fLength    fWidth   fSize   fConc  fConc1     fAsym   fM3Long  \
0      28.7967   16.0021  2.6449  0.3918  0.1982   27.7004   22.0110
1      31.6036   11.7235  2.5185  0.5303  0.3773   26.2722   23.8238
2     162.0520  136.0310  4.0612  0.0374  0.0187  116.7410  -64.8580
3      23.8172    9.5728  2.3385  0.6147  0.3922   27.2107   -6.4633
4      75.1362   30.9205  3.1611  0.3168  0.1832   -5.5277   28.5525
...        ...       ...     ...     ...     ...       ...       ...
19015  21.3846   10.9170  2.6161  0.5857  0.3934   15.2618   11.5245
19016  28.9452    6.7020  2.2672  0.5351  0.2784   37.0816   13.1853
19017  75.4455   47.5305  3.4483  0.1417  0.0549   -9.3561   41.0562
19018 120.5135   76.9018  3.9939  0.0944  0.0683    5.8043  -93.5224
19019 187.1814   53.0014  3.2093  0.2876  0.1539 -167.3125 -168.4558

       fM3Trans   fAlpha     fDist class
0       -8.2027  40.0920   81.8828     g
1       -9.9574   6.3609  205.2610     g
2      -45.2160  76.9600  256.7880     g
3       -7.1513  10.4490  116.7370     g
4       21.8393   4.6480  356.4620     g
...         ...      ...       ...   ...
19015    2.8766   2.4229  106.8258     h
19016   -2.9632  86.7975  247.4560     h
19017   -9.4662  30.2987  256.5166     h
19018  -63.8389  84.6874  408.3166     h
19019   31.4755  52.7310  272.3174     h

[19020 rows x 11 columns]
```

Además de lo solicitado, se realizo una normalización del DataFrame. Para esto se utilizo la función **MinMaxScaler** de **sklearn.preprocessing**. El codigo para la normalización se adjunta en el bloque a continuación.

Código 2: Normalización de los datos.

```
features = magic04_df.drop('class', axis=1)
scaler = MinMaxScaler()
normalized_features = scaler.fit_transform(features)
normalized_df = pd.DataFrame(normalized_features, columns=features.columns)
normalized_df['class'] = magic04_df['class']

print("DataFrame después de la normalización:")
print(normalized_df.head)
```

Lo obtenido mediante la ejecución del bloque anterior se presenta a continuación.

DataFrame después de la normalización:

```
<bound method NDFrame.head of      fLength   fWidth   fSize   fConc  fConc1   fAsym  fM3Long
0      0.074306  0.062415  0.208043  0.430390  0.293229  0.470032  0.620576
1      0.082815  0.045727  0.170668  0.587794  0.558601  0.468649  0.623756
2      0.478241  0.530579  0.626818  0.027617  0.027263  0.556215  0.468201
3      0.059212  0.037338  0.117445  0.683714  0.580679  0.469558  0.570630
4      0.214774  0.120603  0.360674  0.345153  0.271003  0.437870  0.632050
...         ...       ...       ...       ...       ...       ...       ...
19015  0.051838  0.042581  0.199527  0.650756  0.582457  0.457992  0.602182
19016  0.074757  0.026141  0.096363  0.593249  0.412061  0.479112  0.605095
19017  0.215712  0.185389  0.445594  0.146153  0.080901  0.434164  0.653983
19018  0.352326  0.299950  0.606919  0.092397  0.100756  0.448838  0.417922
19019  0.554415  0.206728  0.374926  0.311967  0.227589  0.281277  0.286483

      fM3Trans    fAlpha     fDist class
0      0.512493  0.445467  0.163066     g
1      0.507944  0.070677  0.412679     g
2      0.416540  0.855111  0.516926     g
3      0.515219  0.116100  0.233582     g
4      0.590373  0.051644  0.718582     g
...         ...       ...       ...   ...
19015  0.541215  0.026921  0.213530     h
19016  0.526076  0.964417  0.498046     h
19017  0.509218  0.336652  0.516377     h
19018  0.368263  0.940971  0.823491     h
19019  0.615354  0.585900  0.548344     h

[19020 rows x 11 columns]>
```

Durante el resto de la tarea, se utilizara el Data Frame normalizado (**normalized_df**).

1.1.2. Separación en conjuntos según la clase

En la presente subsección se generan dos conjuntos (Data Frames). Cada conjunto contiene las muestras de una sola clase.
El primer conjunto es el de la clase positiva (Hadrón h), mientras que el segundo conjunto es el de la clase negativa (Gamma g). Para realizar lo señalado, se utilizaron propiedades de la librería pandas.
El código utilizado se adjunta en el bloque a continuación.

Código 3: Creación de Data Frames por clase.

```
hadron_df = normalized_df[normalized_df['class'] == 'h']
gamma_df = normalized_df[normalized_df['class'] == 'g']
print("Subconjunto de la clase hadrón:")
print(hadron_df)
print("-----------------------------------------------")
```

```
6 print("Subconjunto de la clase gamma primarios:")
7 print(gamma_df)
```

Lo obtenido mediante la ejecución del bloque anterior se presenta a continuación.

```
Subconjunto de la clase hadrón:
       fLength   fWidth    fSize    fConc   fConc1    fAsym   fM3Long  \
12332  0.271057  0.147995  0.356032  0.176043  0.149355  0.494768  0.738179
12333  0.296208  0.085816  0.406505  0.106035  0.106831  0.390870  0.657489
12334  0.290985  0.085335  0.345565  0.339698  0.213809  0.396583  0.683098
12335  0.264850  0.073442  0.227203  0.483578  0.506297  0.392227  0.410353
12336  0.102263  0.049432  0.275991  0.449256  0.285227  0.393389  0.596608
...         ...       ...       ...       ...       ...       ...
19015  0.051838  0.042581  0.199527  0.650756  0.582457  0.457992  0.602182
19016  0.074757  0.026141  0.096363  0.593249  0.412061  0.479112  0.605095
19017  0.215712  0.185389  0.445594  0.146153  0.080901  0.434164  0.653983
19018  0.352326  0.299950  0.606919  0.092397  0.100756  0.448838  0.417922
19019  0.554415  0.206728  0.374926  0.311967  0.227589  0.281277  0.286483

       fM3Trans    fAlpha     fDist class
12332  0.564393  0.156916  0.466580     h
12333  0.494704  0.982929  0.553649     h
12334  0.508782  0.230942  0.698291     h
12335  0.489440  0.709816  0.262033     h
12336  0.554398  0.272669  0.328934     h
...         ...       ...       ...   ...
19015  0.541215  0.026921  0.213530     h
19016  0.526076  0.964417  0.498046     h
19017  0.509218  0.336652  0.516377     h
19018  0.368263  0.940971  0.823491     h
19019  0.615354  0.585900  0.548344     h

[6688 rows x 11 columns]
------------------------------------------------
Subconjunto de la clase gamma primarios:
       fLength   fWidth    fSize    fConc   fConc1    fAsym   fM3Long  \
0      0.074306  0.062415  0.208043  0.430390  0.293229  0.470032  0.620576
1      0.082815  0.045727  0.170668  0.587794  0.558601  0.468649  0.623756
2      0.478241  0.530579  0.626818  0.027617  0.027263  0.556215  0.468201
3      0.059212  0.037338  0.117445  0.683714  0.580679  0.469558  0.570630
4      0.214774  0.120603  0.360674  0.345153  0.271003  0.437870  0.632050
...         ...       ...       ...       ...       ...       ...
12327  0.026029  0.044638  0.130041  0.821571  0.563343  0.428610  0.591269
```

```
12328  0.068434  0.080328  0.276198  0.375838  0.318418  0.439876  0.546883
12329  0.053980  0.042495  0.104435  0.596659  0.432064  0.458008  0.613943
12330  0.157439  0.072945  0.292253  0.271054  0.205956  0.536697  0.509529
12331  0.082539  0.075226  0.300562  0.323219  0.224033  0.480178  0.619983

      fM3Trans    fAlpha     fDist class
0     0.512493  0.445467  0.163066     g
1     0.507944  0.070677  0.412679     g
2     0.416540  0.855111  0.516926     g
3     0.515219  0.116100  0.233582     g
4     0.590373  0.051644  0.718582     g
...        ...       ...       ...   ...
12327  0.563883  0.233467  0.410203     g
12328  0.510287  0.044276  0.413159     g
12329  0.552935  0.234089  0.246821     g
12330  0.563250  0.065678  0.396389     g
12331  0.502461  0.195343  0.343823     g

[12332 rows x 11 columns]
```

La utilidad de esta separación radica en la facilidad que brindara para la generación de los histogramas en la subsección siguiente.

1.1.3. Generación de histogramas

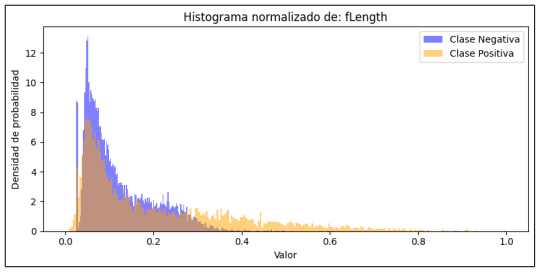
En la presente subsección, se generan los histogramas de todas las características, para todos los datos de las clase negativa y los de la clase positiva.
Para cada característica, los histogramas contendrán los gráficos de ambas clases.
Para la generación de los histogramas se utilizara la función **plt.hist()** con un valor de alpha $= 0.5$. Además los histogramas estarán normalizados y se utilizara una partición según la regla de la raíz cuadrada, en este caso particular, se utilizara un ponderador, por lo que la regla quedara como: Regla $= k * \sqrt{n}$, donde k es el ponderador y n el tamaño del conjunto de datos, en este caso particular se utilizara $k = 3$.
El código utilizado se adjunta en el bloque a continuación.

Código 4: Obtención de histogramas para cada clase.

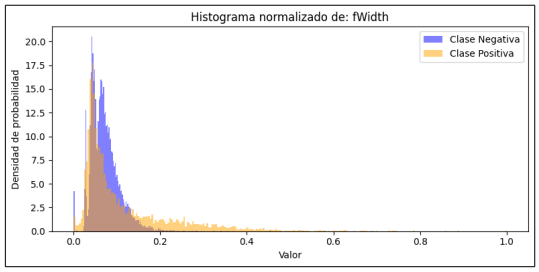
```
# Particiones
bins_gamma = int(3 * np.sqrt(len(gamma_df)))
bins_hadron = int(3 * np.sqrt(len(hadron_df)))
# Histogramas
num_features = len(column_names[0:10])
for i, feature in enumerate(column_names[0:10]):
    fig, axs = plt.subplots(figsize=(8, 4))
    axs.hist(gamma_df[feature], bins=bins_gamma, density=True, alpha=0.5, color='blue',
     ↪ label='Clase Negativa')
```

```
    axs.hist(hadron_df[feature], bins=bins_hadron, density=True, alpha=0.5, color='orange',
     ↪ label='Clase Positiva')
    axs.set_xlabel('Valor')
    axs.set_ylabel('Densidad de probabilidad')
    axs.set_title(f'Histograma normalizado de: {feature}')
    axs.legend(loc='upper right')
    plt.tight_layout()
    plt.show()
```

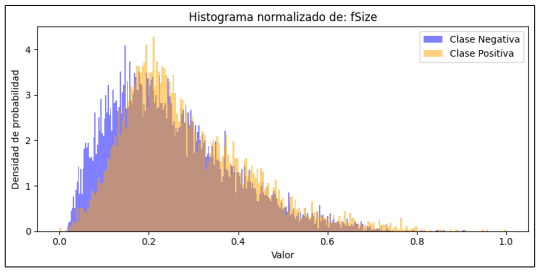
Lo obtenido mediante la ejecución del bloque anterior se presenta a continuación.



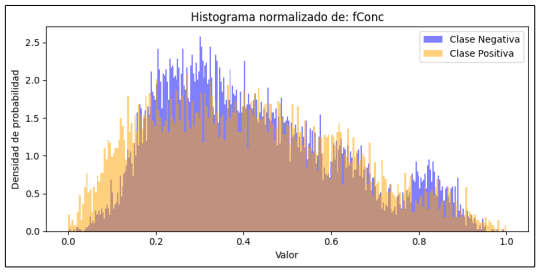
(a) Histograma de fLength.



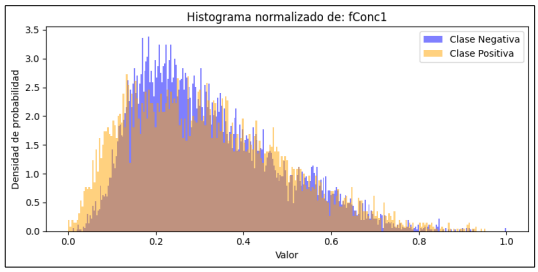
(b) Histograma de fWidth.



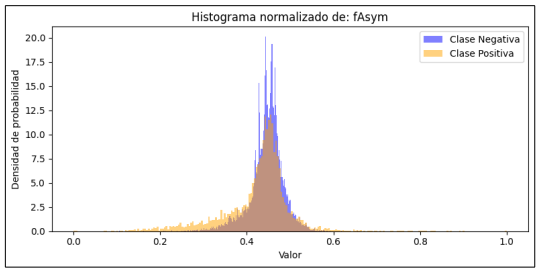
(c) Histograma de fSize.



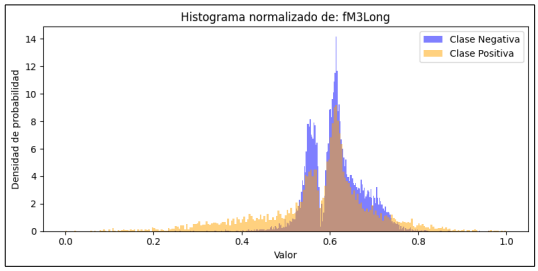
(d) Histograma de fConc.



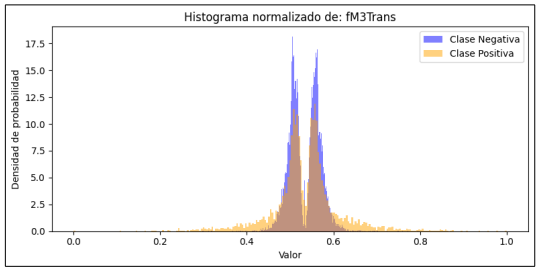
(e) Histograma de fConc1.



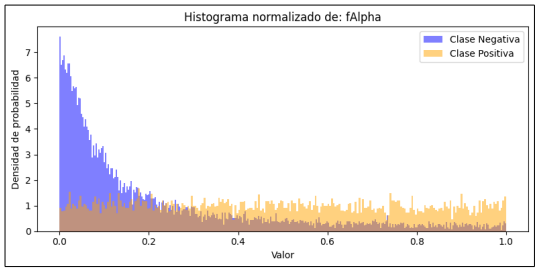
(f) Histograma de fAsym.



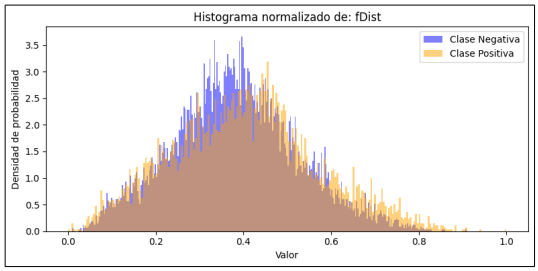
(g) Histograma de fM3Long.



(h) Histograma de fM3Trans.



(i) Histograma de fAlpha.



(j) Histograma de fDist.

Figura 1: Histogramas normalizados de las características con ambas clases separadas.

El análisis de los histogramas se realizara a continuación.

fLength Iniciando con la separación de clases, podemos ver que existe superposición en el rango de valores $[0.0, 0.5]$, aunque a medida que aumenta el valor esta superposición va disminuyendo.
Respecto a la forma de la distribución, se aprecia que estas son similares para ambas clases. Existen picos para ambas clases, siendo los más relevantes los ubicados hacia el inicio, es importante notar que los picos son más pronunciados en la clase negativa.
Además, existen outliers en el caso de la clase negativa.
Por ultimo, es importante señalar que los histogramas parecen tener una distribución Gaussiana.

fWidth Iniciando con la separación de clases, podemos ver que existe superposición en el rango de valores $[0.0, 0.2]$, aunque a medida que aumenta el valor esta superposición va disminuyendo.
Respecto a la forma de la distribución, se aprecia que estas son similares para ambas clases, aunque la clase negativa posee un pico que la positiva no.
Existen picos para ambas clases, siendo los más relevantes los ubicados hacia el inicio, es importante notar que los picos son más pronunciados en la clase negativa.
Además, existen outliers en el caso de la clase negativa.
Por ultimo, es importante señalar que los histogramas parecen tener una distribución Gaussiana.

fSize Iniciando con la separación de clases, podemos ver que existe superposición en el rango de valores $[0.0, 0.7]$.
Respecto a la forma de la distribución, se aprecia que estas son similares para ambas clases, aunque la clase negativa posee mayores valores de densidad de probabilidad.
Existen picos para ambas clases, siendo los más relevantes los ubicados hacia el inicio, en el intervalo de $[0.0, 0.4]$, es importante notar que los picos son más pronunciados en la clase negativa.
Además, existen outliers en el caso de la clase negativa.
Por ultimo, es importante señalar que los histogramas parecen tener una distribución Gaussiana.

fConc Iniciando con la separación de clases, podemos ver que existe superposición en todos los valores ($[0.0, 1.0]$).
Respecto a la forma de la distribución, se aprecia que estas son similares para ambas clases, aunque la clase negativa posee mayores valores de densidad de probabilidad en el rango de $[0.2, 0.4]$, mientras que la clase positiva los posee en el rango de $[0.0, 0.2]$.
Existen picos para ambas clases, siendo los más relevantes los ubicados hacia el inicio, en el intervalo de $[0.0, 0.4]$.
Además, no existen outliers.
Por ultimo, es importante señalar que los histogramas parecen tener una distribución Gaussiana.

fConc1 Iniciando con la separación de clases, podemos ver que existe superposición en los valores en el rango $[0.0, 0.8]$.
Respecto a la forma de la distribución, se aprecia que estas son similares para ambas clases, aunque la clase negativa posee mayores valores de densidad de probabilidad en el rango de $[0.2, 0.4]$, mientras que la clase positiva los posee en el rango de $[0.0, 0.2]$.
Existen picos para ambas clases, siendo los más relevantes los ubicados hacia el inicio, en el intervalo de $[0.0, 0.4]$. Además, la clase negativa tiene outliers.
Por ultimo, es importante señalar que los histogramas parecen tener una distribución Gaussiana.

fAsym Iniciando con la separación de clases, podemos ver que existe superposición en los valores en el rango $[0.2, 0.6]$.
Respecto a la forma de la distribución, se aprecia que estas son similares para ambas clases, y parece ser una distribución con un centro definido, el cual se encuentra alrededor de 0.425.
Existen picos para ambas clases, siendo el más relevante el ubicado en el centro definido previamente.
Además, la clase negativa tiene outliers.
Por ultimo, es importante señalar que los histogramas parecen tener una distribución Gaussiana.

fM3Long Iniciando con la separación de clases, podemos ver que existe superposición en los valores en el rango $[0.4, 0.8]$.
Respecto a la forma de la distribución, se aprecia que estas son similares para ambas clases, y parece ser una distribución con dos picos relevantes, ubicados alrededor de los valores 0.58 y 0.61.
Además, la clase positiva tiene pocos outliers.
Por ultimo, es importante señalar que los histogramas parecen tener una distribución similar a la suma de dos Gaussianas.

fM3Trans Iniciando con la separación de clases, podemos ver que existe superposición en los valores en el rango $[0.4, 0.8]$.
Respecto a la forma de la distribución, se aprecia que estas son similares para ambas clases, y parece ser una distribución con dos picos relevantes, ubicados alrededor de los valores 0.5 y 0.58.
Además, no existen outliers.
Por ultimo, es importante señalar que los histogramas parecen tener una distribución similar a la suma de dos Gaussianas.

fAlpha Iniciando con la separación de clases, podemos ver que existe superposición en todos los valores ($[0.0, 1.0]$).
Respecto a la forma de la distribución, se aprecia que estas no son similares para ambas clases, la clase negativa posee una mayor densidad de probabilidad en el intervalo $[0.0, 0.2]$, mientras que la clase positiva tiene una densidad de probabilidad similar en

todos los valores.
Además, no existen outliers.
Por ultimo, es importante señalar que los histogramas parecen tener una distribución Gaussiana, aunque solo para la clase negativa, la positiva parece tener una distribución uniforme.

fDist Iniciando con la separación de clases, podemos ver que existe superposición en los valores en el rango $[0.0, 0.8]$.
Respecto a la forma de la distribución, se aprecia que estas son similares para ambas clases, la clase negativa posee una mayor densidad de probabilidad en el intervalo $[0.2, 0.4]$, mientras que la clase positiva tiene una mayor densidad de probabilad en el intervalo $[0.4, 0.6]$.
Además, no existen outliers.
Por ultimo, es importante señalar que los histogramas parecen tener una distribución Gaussiana.

Como síntesis: En todas las características existe superposición, las distribuciones son similares para ambas clases en la mayoría de las características y usualmente ocurre que los picos son más pronunciados para la clase negativa. En cuanto a que característica podría ser la más útil o relevante para la clasificación, esta debiese ser fAlpha. Debido a la observación de la distribución Gaussiana (simple y multivariante) se presenta la hipótesis de que el clasificador de Bayes compuesto por Gaussianas Multivariantes, sera superior al compuesto por verosimilitudes de histogramas (el Bayes Ingenuo).

1.1.4. Calculo y representación de la matriz de correlación

En la presente subsección, se calcula la matriz de correlación de las características, utilizando un metodo de los Data Frame de pandas y considerando el valor absoluto de dichas correlaciones.
Se incluirá a la clase en la matriz de correlación, para esto debemos modificar los valores de las clases según que 0 para g y 1 para h, para lo cual se utilizaron métodos de los Data Frame de pandas.
Posterior a esto se gráfico la matriz mediante la función **heatmap()** de seaborn.
El código utilizado se adjunta en el bloque a continuación.

Código 5: Calculo y gráfica de la Matriz de correlación.

```
normalized_df['class'] = normalized_df['class'].replace({'g': 0, 'h': 1})
correlation_matrix = normalized_df.corr().abs()
plt.figure(figsize=(10,8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Matriz de correlación de la data')
plt.show()
```

Lo obtenido mediante la ejecución del bloque anterior se presenta a continuación.

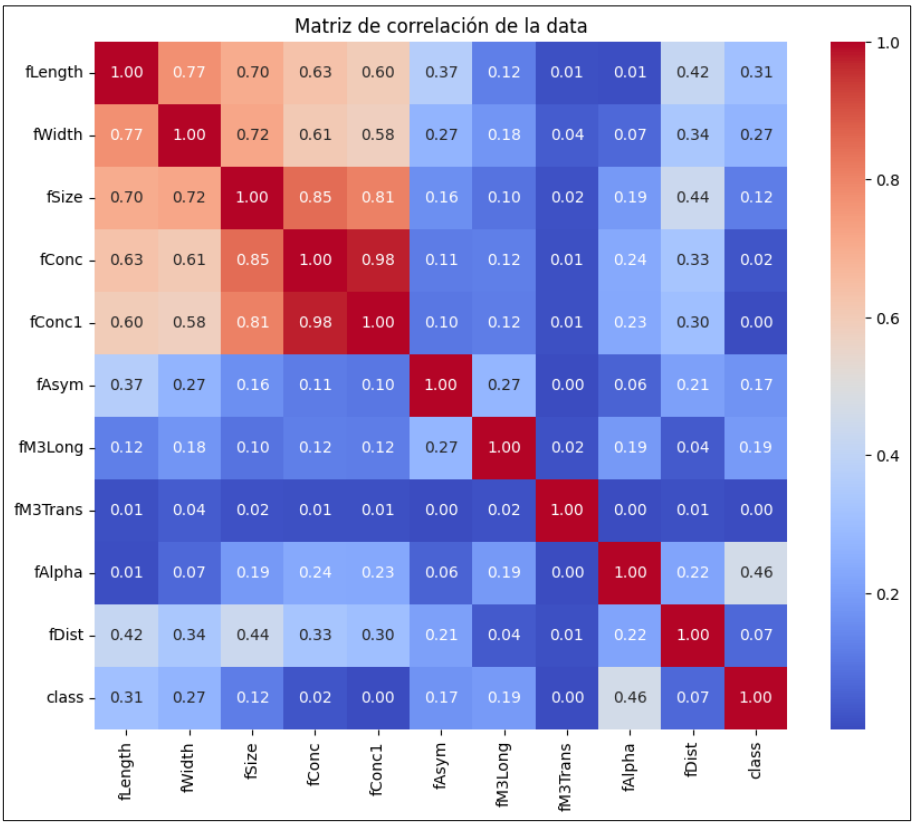


Figura 2: Matriz de correlación de las características.

De la matriz de correlación expuesta arriba, es posible afirmar lo siguiente: Las primeras 5 características son las más correlacionadas entre sí, mientras que el resto de características poseen correlaciones muy bajas.

1.1.5. Orden ascendente de características relacionadas con la clase

En la presente subsección, se obtuvo el orden de las características mas relacionadas con la clase.
Para esto se utilizaron métodos de los Data Frames de pandas.
El código utilizado se adjunta en el bloque a continuación.

Código 6: Orden ascendente de características relacionadas con la clase.

```
class_correlations = correlation_matrix['class']
class_correlations_sorted = class_correlations.sort_values(ascending=False)
class_correlations_sorted = class_correlations_sorted.drop('class')
top_correlations = class_correlations_sorted
print('Data frame con las caracteristicas más correlacionadas con la clase:')
print(top_correlations)
print('------------------------------------------------------------')
print('Las caracteristicas más correlacionadas con la clase:')
```

```
for i, name in enumerate(top_correlations.index.values, start=1):
  print(f'{i}.- {name}')
```

Lo obtenido mediante la ejecución del bloque anterior se presenta a continuación.

```
Data frame con las caracteristicas más correlacionadas con la clase:
fAlpha      0.460979
fLength     0.307572
fWidth      0.265596
fM3Long     0.193409
fAsym       0.173587
fSize       0.117795
fDist       0.065203
fConc       0.024615
fConc1      0.004797
fM3Trans    0.003837
Name: class, dtype: float64
---------------------------------------------------------------
Las caracteristicas más correlacionadas con la clase:
1.- fAlpha
2.- fLength
3.- fWidth
4.- fM3Long
5.- fAsym
6.- fSize
7.- fDist
8.- fConc
9.- fConc1
10.- fM3Trans
```

De la información expuesta anteriormente, es posible afirmar lo siguiente:
La característica más correlacionada con la clase es 'fAlpha', por lo que esta tomara un papel relevante en el clasificador, mientras que la característica menos correlacionada con la clase es 'fM3Trans', por lo que su papel en el clasificador sera menor.

1.1.6. Primeras 3 características más correlacionadas entre sí

En la presente subsección se obtuvieron las primeras 3 características más correlacionadas entre sí, para esto se deberán utilizar métodos de los Data Frame de pandas.
Además, para que no se repitan las correlaciones,es decir, para tener pares únicos de correlaciones se utilizara el triangulo superior de la matriz de correlación calculada en 5. La cual se obtuvo mediante una mascara.
El código utilizado se adjunta en el bloque a continuación.

Código 7: Primeras 3 características más correlacionadas entre sí

```
mask = pd.DataFrame(False, index=correlation_matrix.index, columns=correlation_matrix.
    ↪ columns)
for i in range(len(correlation_matrix.columns)):
  for j in range(i+1, len(correlation_matrix.columns)):
    value = correlation_matrix.iloc[i, j]
    if value not in correlation_matrix.values[:i, j]:
      mask.iloc[i, j] = True
correlation_matrix_unique_values = correlation_matrix.mask(~mask)
correlation_pairs_sorted = correlation_matrix_unique_values.unstack().sort_values(
    ↪ ascending=False).dropna()
top_correlation_pairs = correlation_pairs_sorted[correlation_pairs_sorted !=1].head(3)
print('Data frame con las primeras tres caracteristicas más correlacionadas entre sí:')
print(top_correlation_pairs)
print('------------------------------------------------------------')
print('Primeras tres caracteristicas más correlacionadas entre sí:')
for i, (pair, correlation) in enumerate(top_correlation_pairs.items(), start=1):
  feature1, feature2 = pair
  print(f'{i}.- {feature1} y {feature2}, con una correlación de {correlation}')
```

En el código anterior las primeras 6 líneas generan la mascara, la cual sera utilizada para obtener un Data Frame con los valores únicos de la matriz de correlación. La selección la hace según la primera aparición de los valores.
Lo obtenido mediante la ejecución del bloque anterior se presenta a continuación.

```
Data frame con las primeras tres caracteristicas más correlacionadas entre sí:
fConc1  fConc    0.976412
fConc   fSize    0.850850
fConc1  fSize    0.808835
dtype: float64
------------------------------------------------------------
Primeras tres caracteristicas más correlacionadas entre sí:
1.- fConc1 y fConc, con una correlación de 0.9764118508094626
2.- fConc y fSize, con una correlación de 0.8508497973208063
3.- fConc1 y fSize, con una correlación de 0.8088350117321293
```

De la información expuesta anteriormente, es posible afirmar lo siguiente:
Existe presencia de multicolinealidad en los datos, por lo menos en el caso de las características expuestas.
Es por tanto que se podría reducir alguna de las dos características más correlacionadas, pues la información aportada por una sola podría bastar para el proceso de clasificación, esto debido al alto valor de correlación, es decir, podría llegar a ser redundante el uso de ambas características: **fConc** y **fConc1**.

1.1.7. Gráfico de las dos características más correlacionadas

En la presente subsección se genero el gráfico de dispersión de las dos características más correlacionadas, las cuales corresponden al primer lugar de la lista obtenida en 7. Para la generación de la gráfica se utilizara **plt.scatter()**.
El código utilizado se adjunta en el bloque a continuación.

Código 8: Gráfico de dispersión de las dos características más correlacionadas.

```
feature1, feature2 = top_correlation_pairs.index[0]
feature1_data = normalized_df[feature1]
feature2_data = normalized_df[feature2]

plt.figure(figsize=(8,6))
plt.scatter(feature1_data, feature2_data, alpha=0.5)
plt.xlabel(feature1)
plt.ylabel(feature2)
plt.title(f'Grafico de dispersión de {feature1} vs {feature2}')
plt.grid(True)
plt.show()
```

Lo obtenido mediante la ejecución del bloque anterior se presenta a continuación.

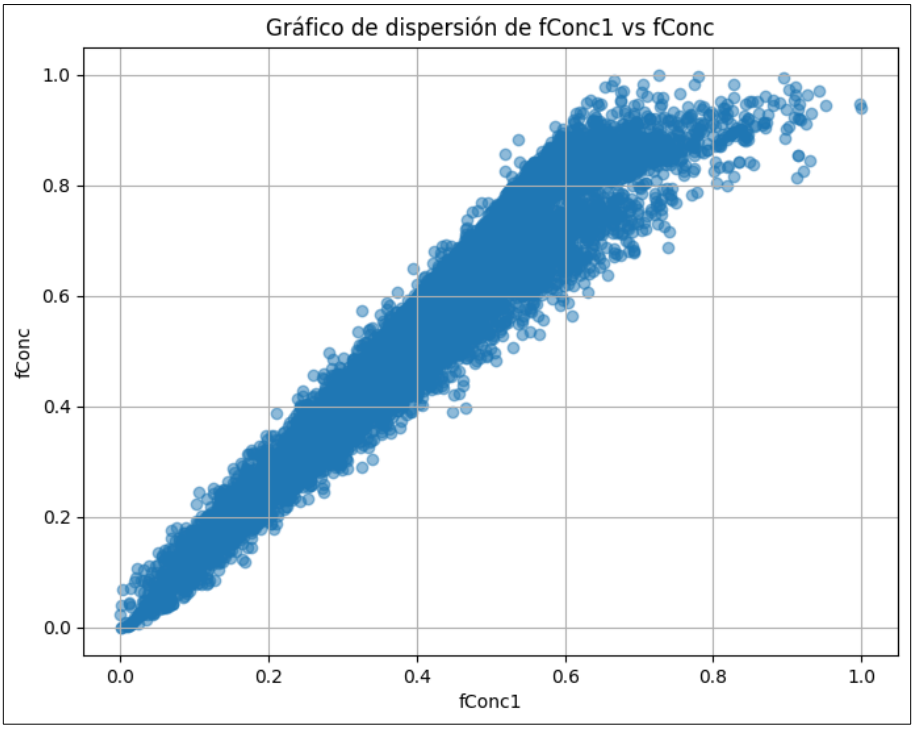


Figura 3: Gráfico de dispersión de las dos características más correlacionadas entre sí.

A partir del gráfico anterior es posible afirmar lo siguiente:
Existe una clara tendencia o patrón de dispersión, pues los puntos tienden a formar una linea diagonal.
Debido a lo anterior se comprueba que existe una correlación lineal positiva fuerte entre las características.
Aún así, existen algunos outliers, como por ejemplo los puntos cercanos al $(1.0, 1.0)$.

1.1.8. Ultimas 3 características más correlacionadas entre sí

En la presente subsección se obtuvieron las ultimas 3 características más correlacionadas entre sí, para esto se deberán utilizar métodos de los Data Frame de pandas.
Además, para que no se repitan las correlaciones se utilizara el triangulo superior de la matriz de correlación calculada en 7.
El código utilizado se adjunta en el bloque a continuación.

Código 9: Ultimas 3 características más correlacionadas entre sí.

```
last_correlation_pairs = correlation_pairs_sorted[(correlation_pairs_sorted !=1)].
    ↪ sort_values(ascending=True).head(3)
print('Data frame con las ultimas tres caracteristicas más correlacionadas entre sí:')
print(last_correlation_pairs)
print('------------------------------------------------------------')
print('Ultimas tres caracteristicas más correlacionadas entre sí:')
for i, (pair, correlation) in enumerate(last_correlation_pairs.items(), start=1):
  feature1, feature2 = pair
  print(f'{i}.- {feature1} y {feature2}, con una correlación de {correlation}')
```

Lo obtenido mediante la ejecución del bloque anterior se presenta a continuación.

```
Data frame con las ultimas tres caracteristicas más correlacionadas entre sí:
fM3Trans  fAsym       0.002553
class     fM3Trans    0.003837
fAlpha    fM3Trans    0.004659
dtype: float64
------------------------------------------------------------
Ultimas tres caracteristicas más correlacionadas entre sí:
1.- fM3Trans y fAsym, con una correlación de 0.002552837319858875
2.- class y fM3Trans, con una correlación de 0.0038371980687589265
3.- fAlpha y fM3Trans, con una correlación de 0.004659125870349372
```

A partir de lo anterior es posible afirmar lo siguiente:
Las correlaciones son muy débiles, pues son cercanas a 0.
Además, la correlación entre la clase y la característica fM3Trans es muy baja, por lo que quizás seria una buena idea eliminar fM3Trans en el caso puntual de esta tarea, es decir, en el caso de un clasificador de Bayes.

1.1.9. Gráfico de las dos características menos correlacionadas

En la presente subsección se genero el gráfico de dispersión de las dos características menos correlacionadas, las cuales corresponden al primer lugar de la lista obtenida en 9.
Para la generación del gráfico se utilizo **plt.scatter()**.
El código utilizado se adjunta en el bloque a continuación.

Código 10: Gráfico de dispersión de las dos características menos correlacionadas.

```
feature1, feature2 = last_correlation_pairs.index[0]
feature1_data = normalized_df[feature1]
feature2_data = normalized_df[feature2]

plt.figure(figsize=(8,6))
plt.scatter(feature1_data, feature2_data, alpha=0.5)
plt.xlabel(feature1)
plt.ylabel(feature2)
plt.title(f'Gráfico de dispersión de {feature1} vs {feature2}')
plt.grid(True)
plt.show()
```

Lo obtenido mediante la ejecución del bloque anterior se presenta a continuación.

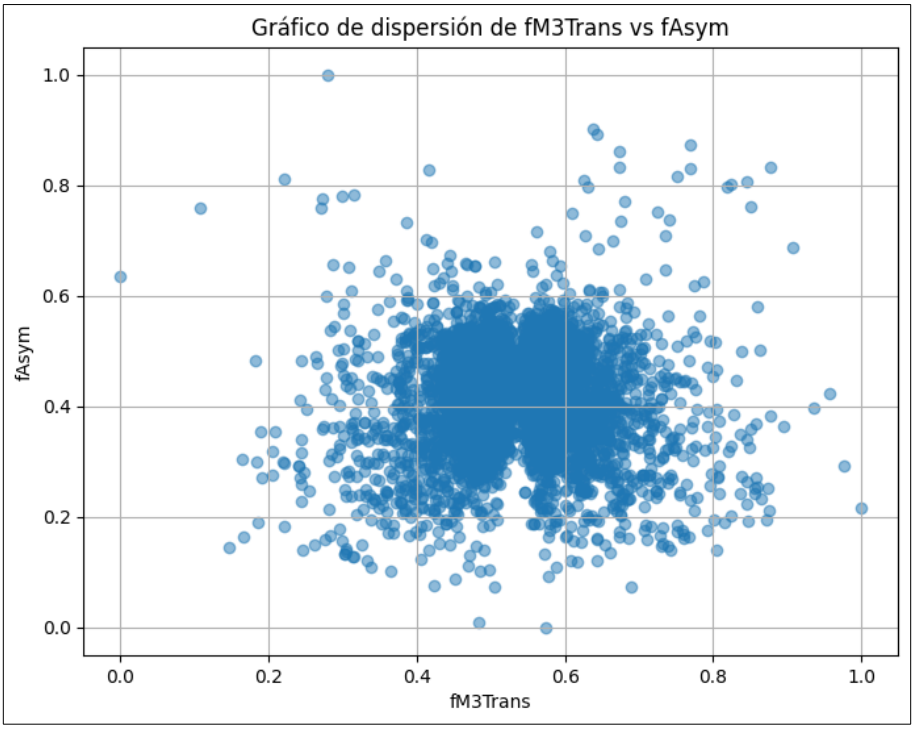


Figura 4: Gráfico de dispersión de las dos características menos correlacionadas entre sí.

A partir del gráfico anterior es posible afirmar lo siguiente:
Como la correlación entre las características es muy baja, el gráfico presenta una dispersión aleatoria, en contraste con el gráfico 3, en donde si existe un patrón lineal.
Además, en el caso de 4, es posible notar que existen agrupaciones de puntos en los intervalos: fM3Trans $[0.4, 0.7]$ y fAsym $[0.2, 0.6]$, lo cual indica que quizás la relación entre ambas características es no lineal.
Por ultimo señalar que existen varios outliers, los cuales corresponden a los puntos que se encuentran alejados del cumulo central.

1.2. Clasificación usando Naive Bayes e histogramas

En la presente subsección se programo, entreno y calibro un clasificador Bayes Ingenuo, utilizando como verosimilitudes los histogramas.
La regla de decisión del mencionado clasificador está dada por:

$$\frac{P(\text{características}|\text{positiva})}{P(\text{características}|\text{negativa})} \geq \Theta \tag{1}$$

Ecuación del clasificador Naive Bayes.

En donde Θ representa al umbral y depende de los costos asociados a cada decisión ($c_{positiva}$ y $c_{negativa}$) y de las probabilidades a priori, las cuales son estimadas mediante los histogramas.
La ecuación que define al umbral es:

$$\Theta = \frac{c_{\text{positiva}} P(\text{negativa})}{c_{\text{negativa}} P(\text{positiva})} \tag{2}$$

Ecuación del umbral.

En el caso de la tarea no se utilizara esta ecuación, si no que utilizaran valores en el rango de $[0, \infty]$, aun así, dicha ecuación sera implementada en código.

1.2.1. Dividir Data Set en conjuntos de entrenamiento y prueba

En la presente subsección se dividio al Data Frame que contiene al Data Set en 2 conjuntos representativos.
Estos serán los conjuntos de entrenamiento y prueba. El primero contendrá el 80 % de los datos, mientras que el segundo contendrá el 20 % de los datos.
Además, se mantendrá la proporcion entre las clases del conjunto original.
Para realizar la separación se utilizara la funcion **train_test_split()**, usando un **random_state** fijo.
El código utilizado para realizar lo anterior se adjunta en el bloque a continuación.

Código 11: Separación del Data Set en 2 conjuntos representativos (Entrenamiento y Prueba).

```
1 X = normalized_df.drop('class', axis=1)
2 y = normalized_df['class']
```

```
3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
    ↪ random_state=3)
4 original_proportion = [round(len(gamma_df)/len(magic04_df), 4), round(len(hadron_df)/
    ↪ len(magic04_df), 4)]
5 train_proportion = [round((y_train==0).sum()/len(y_train), 4), round((y_train==1).sum
    ↪ ()/len(y_train), 4)]
6 test_proportion = [round((y_test==0).sum()/len(y_test), 4), round((y_test==1).sum()/
    ↪ len(y_test), 4)]
7 print(original_proportion, train_proportion, test_proportion)
```

En el código anterior se realizo lo solicitado y para mantener la proporción entre las clases se utiliza `stratify=y`.
Además se utiliza `random_state=3`.
Por ultimo, se utilizan las líneas [4, 7] para calcular la proporción entre las clases del conjunto: Original (4), Entrenamiento (5) y Prueba (6), mientras que la ultima linea (7) despliega los valores de dicha proporción. Gracias a eso, se obtiene lo siguiente.

```
[0.6484, 0.3516] [0.6484, 0.3516] [0.6483, 0.3517]
```

De lo anterior, podemos afirmar que la proporción entre las clases se mantiene casi exacta. Solo en el caso del conjunto de prueba los valores cambian, disminuyendo en una millonésima la proporción de la clase **g** y aumentando en una millonésima la proporción de la clase **h**, por lo cual es posible afirmar que la proporción se mantiene.
Como comentario final, es posible que la causa de lo anterior sea el hecho de que las muestras por clase no son valores factores de 10, p. ej. 12330 y 6690.

1.2.2. Implementación de un clasificador Naive Bayes con histogramas por verosimilitudes

En la presente subsección, se programaron varias funciones, para finalmente implementar un clasificador Bayes Ingenuo que utilice histogramas para representar las verosimilitudes. Es importante recalcar que se programaron 5 funciones, estas se listan a continuación.

train_naive_bayes Es la función que entrena al clasificador.
Recibe como entrada las características contenidas en X_{train}, las clases contenidas en y_{train} y el numero de bins.
Retorna la probabilidad a priori de que una instancia pertenezca a alguna clase y los bins de características para cada clase.

find_bin_index Es la función que encuentra el índice del bin al que pertenece un valor.
Retorna el índice del bin al que pertenece el valor.

calculate_likelihood Es la función que calcula la verosimilitud.
Recibe como entrada una instancia, los bins de características y un valor de épsilon (preseteado en 1×10^{-9}).
Retorna el valor de verosimilitud de la instancia en particular.

predict_naive_bayes Es la función que clasifica, es decir, es el clasificador Bayes Ingenuo solicitado.
Recibe como entrada las características contenidas en X_{test}, las clases a priori, los bins de características y el umbral.
Retorna las clases seleccionadas por el clasificador.

calculate_theta Es la función que calcula el valor de theta, el cual se debiese utilizar como umbral en la función anterior.
Finalmente no es utilizada.

La necesidad de programar tantas funciones es debido a que no se pueden utilizar funciones de librerías.
El código completo se adjunta en el bloque a continuación.

Código 12: Implementación de un clasificador Bayesiano Ingenuo con histogramas por verosimilitudes.

```
bins = int(3 * np.sqrt(len(X_test)))

def train_naive_bayes(X, y, bins=bins):
  classes = [0, 1]
  assert len(classes) == 2
  class_priors = {}
  feature_bins = {}
  for cls in classes:
      cls_samples = X[y == cls]
      class_priors[cls] = len(cls_samples)/len(y)
      feature_bins[cls] = []
      for col in X.columns:
          hist, bin_edges = np.histogram(cls_samples[col], bins=bins, density=True)
          hist_normalized = hist / sum(hist)
          feature_bins[cls].append((hist_normalized, bin_edges))

  return class_priors, feature_bins

def find_bin_index(bin_edges, value):
  for i in range(len(bin_edges) - 1):
    if bin_edges[i] <= value < bin_edges[i + 1]:
      return i
  return -1

def calculate_likelihood(instance, feature_bins, epsilon=1e-9):
    likelihood = 1
    for index, hist_and_edges in enumerate(feature_bins):
        hist, bin_edges = hist_and_edges
        bin_index = find_bin_index(bin_edges, instance[index])
        if bin_index >= 0 and bin_index < len(hist) - 1:
            left_bin = hist[bin_index]
```

```
            right_bin = hist[bin_index + 1]
            bin_width = bin_edges[1] - bin_edges[0]
            interpolation_factor = (instance[index] - bin_edges[bin_index]) / bin_width
            left_bin_smoothed = left_bin + epsilon
            right_bin_smoothed = right_bin + epsilon
            likelihood *= left_bin_smoothed + (right_bin_smoothed - left_bin_smoothed) *
    ↪ interpolation_factor
        else:
            likelihood *= epsilon

    return likelihood


def predict_naive_bayes(X, class_priors, feature_bins, theta):
    predictions = []
    for _, instance in X.iterrows():
      likelihood_0 = calculate_likelihood(instance, feature_bins[0])
      likelihood_1 = calculate_likelihood(instance, feature_bins[1])
      prob_ratio = (class_priors[1] * likelihood_1) / (class_priors[0] * likelihood_0)
      predicted_class = 1 if prob_ratio >= theta else 0
      predictions.append(predicted_class)

    return predictions

def calculate_theta(c_pos, p_pos, c_neg, p_neg):
  return (c_pos * p_neg) / (c_neg * p_pos)
```

Además de todo lo mencionado anteriormente en el listado, en este bloque de código se define el valor de los bins a utilizar, el cual es: `bins = int(3 * np.sqrt(len(X_test)))`. Es decir, se utiliza la regla de la raíz cuadrada previamente mencionada.

1.2.3. Calculo de histogramas para cada clase

En la presente subsección se implementa una función que permite calcular los histogramas normalizados de cada clase a partir de X_{train}.
El código con la implementación se presenta en el siguiente bloque.

Código 13: Función que permite obtener los histogramas normalizados de cada clase.

```
def calculate_class_histograms(X, y, bins=bins):
  classes = [0, 1]
  class_histograms = {}
  for cls in classes:
    cls_samples = X[y == cls]
    class_histograms[cls] = []
    for col in X.columns:
      hist, bin_edges = np.histogram(cls_samples[col], bins=bins, density=True)
```

```
        hist_normalized = hist / sum(hist)
        class_histograms[cls].append((hist_normalized, bin_edges))

    return class_histograms

class_histograms = calculate_class_histograms(X_train, y_train)
```

En el código anterior se implementa lo solicitado y se almacenan los histogramas en la variable class_histograms.
Además, se normalizan los histogramas.
Por ultimo mencionar que la función retorna una lista de tuplas, en donde las tuplas contienen el histograma normalizado y los bins de características para cada clase, en ese orden.

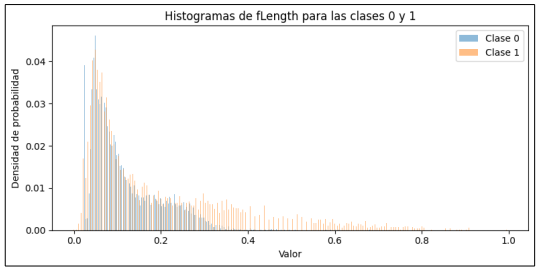
1.2.4. Generación de histogramas

En la presente subsección se generan los histogramas de cada característica para ambas clases con la función **plt.bar()**, utilizando **alpha=0.5**.
El código utilizado para lograr lo solicitado se adjunta en el bloque a continuación.

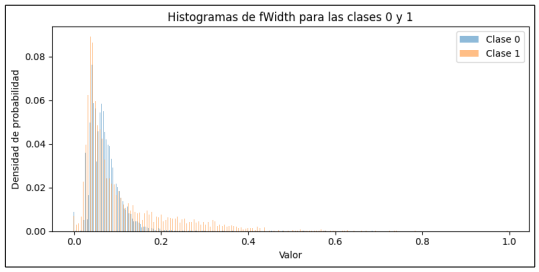
Código 14: Obtención de histogramas para cada clase 2.

```
classes = [0, 1]
num_features = len(X_train.columns)
num_classes = len(classes)
for i, col in enumerate(X_train.columns):
    hist0, bin_edges0 = class_histograms[classes[0]][i]
    hist1, bin_edges1 = class_histograms[classes[1]][i]
    bar_width = (bin_edges0[1] - bin_edges0[0]) * 0.4
    fig, axs = plt.subplots(figsize=(8, 4))
    axs.bar(bin_edges0[:-1], hist0, width=bar_width, alpha=0.5, label=f'Clase {classes[0]}')
    axs.bar(bin_edges1[:-1], hist1, width=bar_width, alpha=0.5, label=f'Clase {classes[1]}')
    axs.set_xlabel('Valor')
    axs.set_ylabel('Densidad de probabilidad')
    axs.set_title(f'Histogramas de {col} para las clases {classes[0]} y {classes[1]}')
    axs.legend()
    plt.tight_layout()
    plt.show()
```

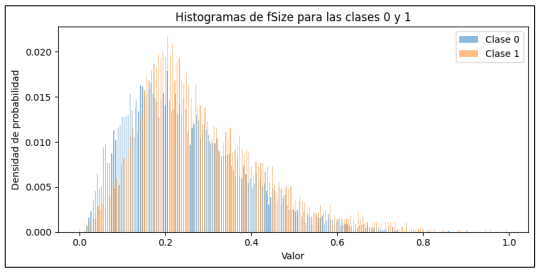
Lo obtenido mediante la ejecución del bloque anterior se presenta a continuación.



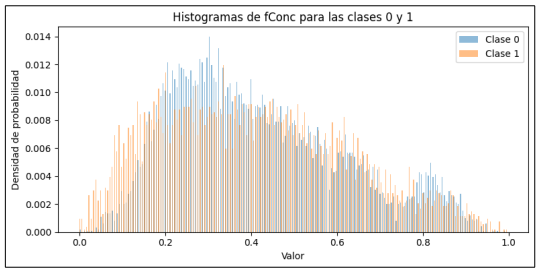
(a) Histograma de fLength.



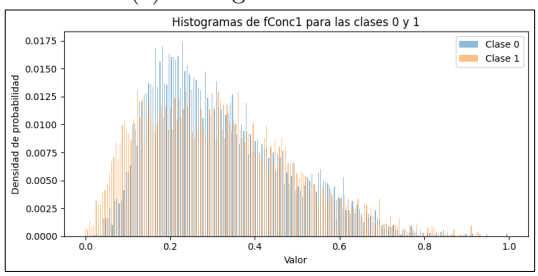
(b) Histograma de fWidth.



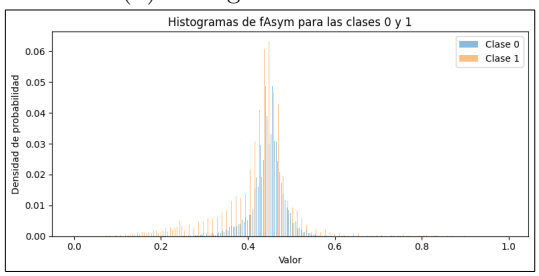
(c) Histograma de fSize.



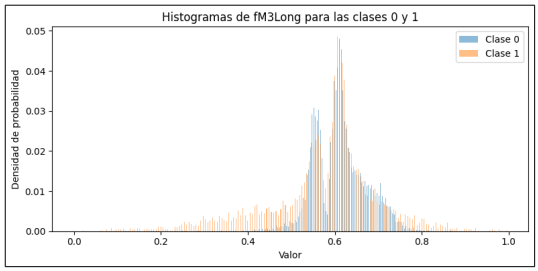
(d) Histograma de fConc.



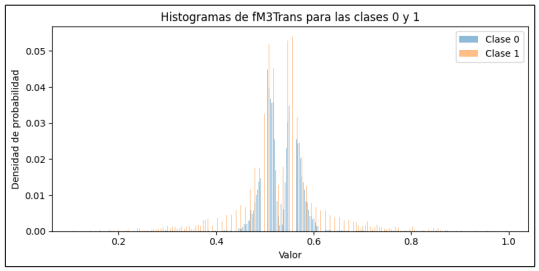
(e) Histograma de fConc1.



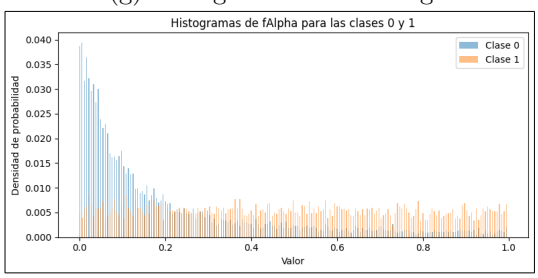
(f) Histograma de fAsym.



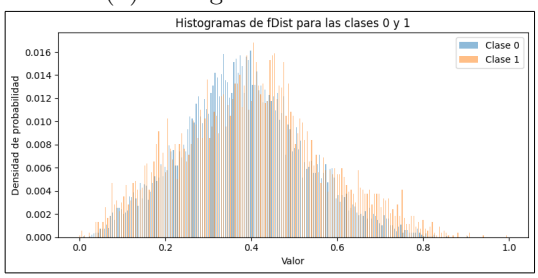
(g) Histograma de fM3Long.



(h) Histograma de fM3Trans.



(i) Histograma de fAlpha.



(j) Histograma de fDist.

Figura 5: Histogramas normalizados de las características con ambas clases separadas.

Es interesante notar que los histogramas obtenidos en esta subsección se asemejan a los obtenidos en la subsección 1.1.3, por lo que el análisis descrito en 1.1.3 es igualmente aplicable en este, con la salvedad de que en este caso se observan más espacios vacíos en los histogramas. Esto sucede debido a que estos se realizaron con el conjunto de entrenamiento, el cual consiste en el 80 % de las muestras tomadas de manera aleatoria.

1.2.5. Curva ROC

En la presente subsección se realizara una prueba del clasificador.
Para lo anterior, se utilizaran distintos umbrales (valores de Θ) y se realizara la clasificación del conjunto de prueba.
Además se obtendra la Tasa de Verdaderos Positivos y la Tasa de Falsos Positivos. Con lo anterior se generara la curva ROC para el conjunto de prueba.
Es importante recalcar que se programaron dos funciones nuevas, estas se listan a continuación.

confusion_matrix Es la función que calcula la matriz de confusión.
Recibe como entrada el conjunto que contiene clases verdaderas (en nuestro caso, las del conjunto **y_test**) y el conjunto que contiene las clases obtenidas mediante el uso de los clasificadores diseñados (**Naive Bayes** y **Gaussian Multivariates**).
Retorna los valores de la matriz de confusión (en este caso, de tamaño dos por dos), los valores entregados son `tn, fp, fn, tp`, lo que corresponde a los valores de: **True Negative**, **False Positive**, **False Negative** y **True positive**.

calculate_TVP_and_TFP Es la función que calcula la Tasa de Verdaderos Positivos y la Tasa de Falsos Positivos.
Recibe como entrada el conjunto que contiene clases verdaderas (en nuestro caso, las del conjunto **y_test**) y el conjunto que contiene las clases obtenidas mediante el uso de los clasificadores diseñados (**Naive Bayes** y **Gaussian Multivariates**).
Retorna los valores previamente señalados (TVP y TFP).

La necesidad de programar las funciones anteriores es debido a que no se pueden utilizar funciones de librerías.
El código completo se adjunta en el bloque a continuación.

Código 15: Generación de la curva ROC Bayes Ingenuo.

```
def confusion_matrix(y_true, y_predict):
  matrix = [[0, 0], [0, 0]]
  for true, pred in zip(y_true, y_predict):
    matrix[true][pred] += 1

  return matrix[0][0], matrix[0][1], matrix[1][0], matrix[1][1]

def calculate_TVP_and_TFP(y, y_pred):
  tn, fp, fn, tp = confusion_matrix(y, y_pred)
  TVP = tp / (tp + fn) if (tp + fn) != 0 else 0
```

```
  TFP = fp / (fp + tn) if (fp + tn) != 0 else 0

  return TVP, TFP

class_priors, feature_bins = train_naive_bayes(X_train, y_train)
thresholds = np.concatenate((np.linspace(0, 1, 10), np.logspace(np.log10(1), np.log10(1000),
    ↪ 10)))
tvp_points = []
tfp_points = []
predictions_list = []
for theta in thresholds:
  predictions = predict_naive_bayes(X_test, class_priors, feature_bins, theta)
  tvp, tfp = calculate_TVP_and_TFP(y_test.values, predictions)
  tvp_points.append(tvp)
  tfp_points.append(tfp)
plt.figure(figsize=(8, 6))
plt.plot(tfp_points, tvp_points, marker='o', color='blue', label='ROC')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
plt.xlabel('Tasa de Falsos Positivos (TFP)')
plt.ylabel('Tasa de Verdaderos Positivos (TVP)')
plt.title('Curva ROC Naive Bayes')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```

Es importante decir que la lista de umbrales utilizadas es:

[0.0000e+00 1.1110e-01 2.2220e-01 3.3330e-01 4.4440e-01 5.5560e-01
6.6670e-01 7.7780e-01 8.8890e-01 1.0000e+00 1.0000e+00 2.1544e+00
4.6416e+00 1.0000e+01 2.1544e+01 4.6415e+01 1.0000e+02 2.1544e+02
4.6415e+02 1.0000e+03]

A continuación se adjunta lo obtenido mediante el código anterior.

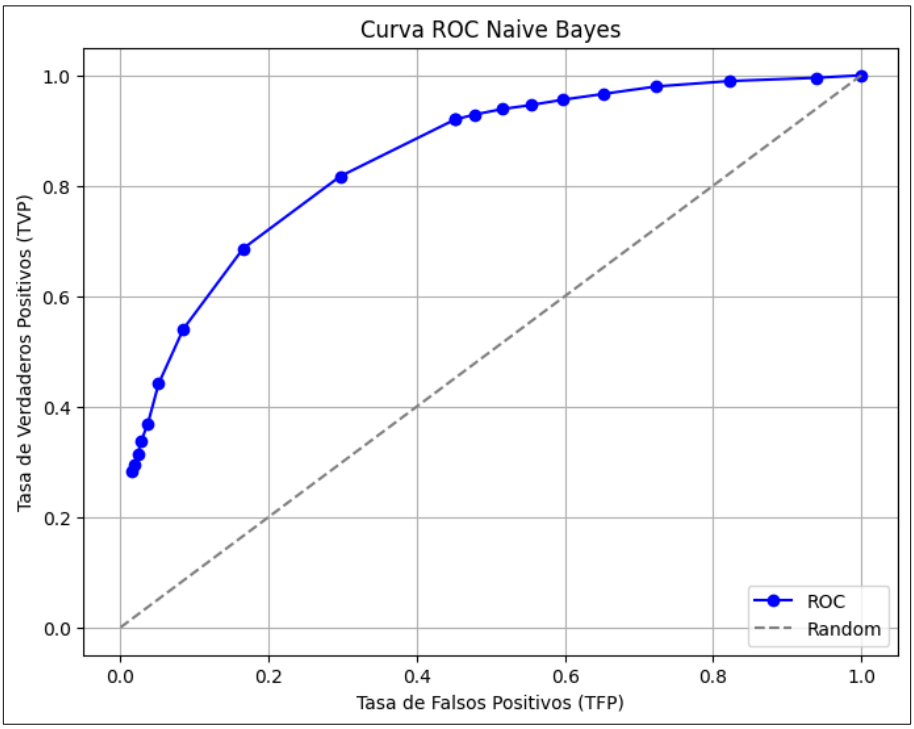


Figura 6: Curva ROC Naive Bayes.

A continuación se realizara el análisis del gráfico 6. Es de suma relevancia notar que el clasificador, al menos con los umbrales utilizados, nunca alcanza el punto $(0, 0)$, por lo cual se puede intuir que este no predice todas las instancias como negativas en ningún caso. Además, como el clasificador alcanza el punto $(1, 1)$ con el umbral igual a cero, por lo cual se puede intuir que este predice todas las instancias como positivas.
Además, es posible observar que en la medida que aumenta el valor del umbral, mejor comportamiento tiene el clasificador, hasta cierto punto (cercano a $(0.4, 0.9)$), desde donde comienza a decaer drásticamente el valor de **TVP**.
También, la curva generada se aleja bastante de la diagonal, que representa a un clasificador Random, por lo que es posible inferir que el clasificador implementado obtiene buenos resultados.
Por ultimo, como la curva generada esta por encima de la línea Random, se puede afirmar que este clasificador es mejor que el azar.

1.2.6. Curva Precision-Recall

En la presente subsección se genera la curva Precision-Recall. Para lo anterior, se utilizaran los mismos umbrales que en la subsección previa, con los cuales se realizara la clasificación del conjunto de prueba. Además, se obtendrán los valores de Precision y Recall.
Con lo anterior se generara la curva Precision-Recall para el conjunto de prueba.
Es importante recalcar que se programo una función nueva, la cual se describe a continuación.

calculate_precision_and_recall Es la función que calcula los valores de Precision y

Recall. Recibe como entrada el conjunto que contiene clases verdaderas (en nuestro caso, las del conjunto **y_test**) y el conjunto que contiene las clases obtenidas mediante el uso de los clasificadores diseñados (**Naive Bayes** y **Gaussian Multivariates**). Retorna los valores previamente señaladas (Precision y Recall).

La razón por la que programar la función anterior es debido a que no se pueden utilizar funciones librerías.
El código completo se adjunta en el bloque a continuación.

Código 16: Generación de la curva Precision-Recall Naive Bayes.

```
def calculate_precision_and_recall(y, y_pred):
  tn, fp, fn, tp = confusion_matrix(y, y_pred)
  precision = tp / (tp + fp) if tp + fp != 0 else 0
  recall = tp / (tp + fn) if tp + fn != 0 else 0
  return precision, recall

class_priors, feature_bins = train_naive_bayes(X_train, y_train)
thresholds = np.concatenate((np.linspace(0, 1, 10), np.logspace(np.log10(1), np.log10(1000),
    ↪ 10)))
precision_points = []
recall_points = []
f1_scores = []
for theta in thresholds:
  predictions = predict_naive_bayes(X_test, class_priors, feature_bins, theta)
  precision, recall = calculate_precision_and_recall(y_test.values, predictions)
  precision_points.append(precision)
  recall_points.append(recall)
  f1_scores.append(2 * (precision * recall) / (precision + recall) if (precision + recall) != 0
    ↪ else 0)
best_performance_index = np.argmax(np.array(f1_scores))
best_precision = precision_points[best_performance_index]
best_recall = recall_points[best_performance_index]
best_threshold = thresholds[best_performance_index]
plt.figure(figsize=(8, 6))
plt.plot(recall_points, precision_points, marker='o', color='blue', label='Precision-Recall')
plt.plot(best_recall, best_precision, marker='o', color='orange', label=f'Mejor desempeño (
    ↪ Theta={best_threshold:.2f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Curva Precision-Recall Naive Bayes')
plt.grid(True)
plt.legend()
plt.show()
```

Además de lo solicitado, se ha agregado el calculo del mejor umbral, el cual es obtenido mediante la maximización de los **f1_scores**.
A continuación se adjunta lo obtenido mediante el código anterior.

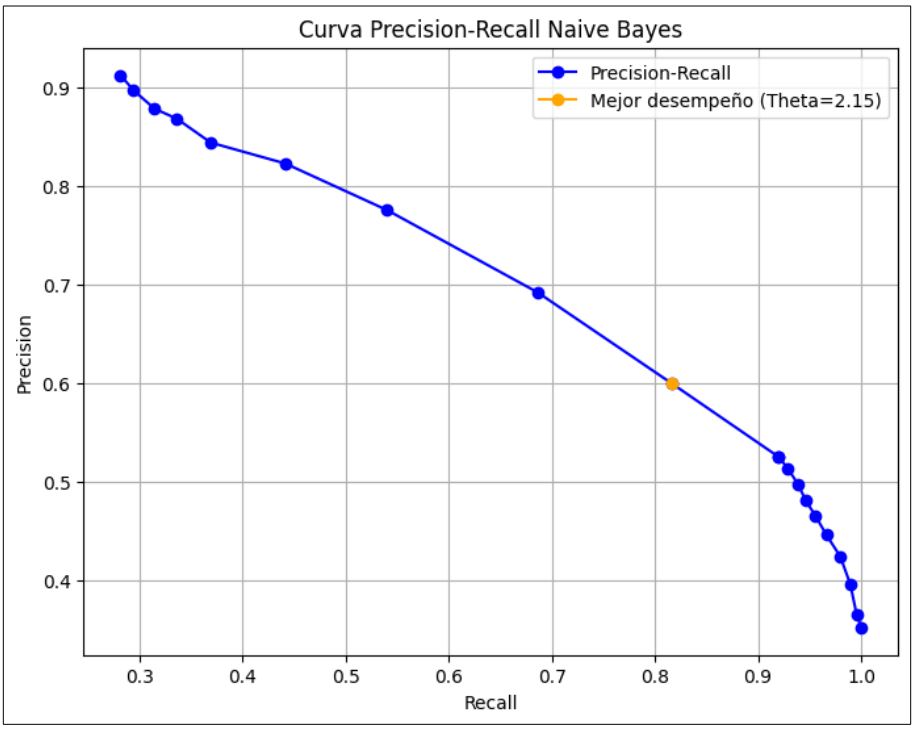


Figura 7: Curva Precision-Recall Naive Bayes.

A continuación se realizara el análisis del gráfico 7.
Es relevante notar que en el gráfico generado se puede visualizar claramente el Trade-off entre Precision y Recall, esto sucede a medida que aumenta el valor del umbral y consiste en que a medida que aumenta la Precision, el recall tiende a disminuir.
Para localizar el punto óptimo se utilizo la métrica F1-score, con lo cual se obtuvo el umbral $\Theta = 2.15$, mediante dicho umbral se obtuvo el punto señalado en naranja, el cual representa un equilibrio entre Precision y Recall.
Por ultimo, se adjuntara la lista de valores de F1-score.

```
[0.5204200700116686, 0.5341888911169039, 0.5648464163822525,
0.5914730430859463, 0.6099056603773586, 0.6252749938890246,
0.6376227650465877, 0.650103519668737, 0.6609944163786227,
0.6684767852294325, 0.6684767852294325, 0.6913345983554713,
0.6891891891891891, 0.6370044052863436, 0.5749027237354085,
0.5137805512220489, 0.4849137931034483, 0.4634012107870116,
0.44344400675295437, 0.4306110793832096]
```

El análisis de la lista anterior se realizara a continuación.
Se logra observar una tendencia, en los rangos iniciales se observa un crecimiento, hasta

obtener un valor máximo 0.6913345983554713, posterior a eso los valores comienzan a decaer. Notar que si bien este es el óptimo en este intervalo, cabe la posibilidad de que exista otro superior, es decir, este es el óptimo local, no global.

1.3. Clasificación usando Gaussianas Multivariantes

En esta subsección se programo un clasificador Bayesiano basado en gaussianas multivariantes, el cual no utiliza la aproximación Naive Bayes.

1.3.1. Entrenar un Modelo Gaussiano multidimensional para cada clase

En la presente subsección se implemento un código que realiza el entrenamiento de un modelo gaussiano multidimensional, calculando la media y covarianza, a partir del conjunto de entrenamiento.
El código mencionado se adjunta en el siguiente bloque.

Código 17: Entrenador Gaussiano Multivariantes.

```
def train_gaussian(X, y):
  classes = [0, 1]
  class_gaussians = {}
  for cls in classes:
    cls_samples = X[y == cls]
    mean = np.mean(cls_samples, axis=0)
    covariance = np.cov(cls_samples, rowvar=False)
    class_gaussians[cls] = {'mean': mean, 'covariance': covariance}

  return class_gaussians

class_gaussians = train_gaussian(X_train, y_train)
for cls, gaussian in class_gaussians.items():
  print(f'Clase {cls}:')
  print(f'\n Media: {gaussian["mean"]}')
  print(f'\n Covarianza: {gaussian["covariance"]}')
```

Como se puede observar en el bloque anterior, la función de entrenamiento implementada tiene por entrada X (el conjunto de características) e y (el conjunto de las clases).
La función crea un diccionario vació, llamado **class_gaussians**, luego itera sobre la lista de clases, con lo que obtienen las **cls_samples**, que consisten en las muestras del conjunto de datos X que pertenecen a una clase especifica.
Posteriormente se calcula la media y covarianza de **cls_samples**.
A continuación se llena el diccionario con la llave como los valores de las clases y como valor un diccionario que contiene la media y covariancia calculadas previamente.
Finalmente, la función retorna el diccionario **class_gaussians**.
Ahora, se adjuntan los resultados del **print()**.

Clase 0:

```
 Media: fLength      0.119331
fWidth       0.072558
fSize        0.249018
fConc        0.420498
fConc1       0.317963
fAsym        0.446400
fM3Long      0.613295
fM3Trans     0.534345
fAlpha       0.207543
fDist        0.383088
dtype: float64

 Covarianza: [[ 6.26400444e-03  1.94804573e-03  8.25757777e-03 -1.13477844e-02
  -8.44527492e-03 -3.53377945e-04  2.54265382e-03  2.88218529e-05
  -8.32608753e-03  6.73070402e-03]
 [ 1.94804573e-03  1.24666541e-03  4.09996436e-03 -5.39847433e-03
  -4.03476116e-03 -3.84891773e-06  8.82100946e-04  4.70211184e-05
  -2.47559640e-03  1.96192447e-03]
 [ 8.25757777e-03  4.09996436e-03  1.85689345e-02 -2.36866080e-02
  -1.76198909e-02  1.78410386e-04  4.39510798e-03  8.68362112e-05
  -1.48658217e-02  8.92339506e-03]
 [-1.13477844e-02 -5.39847433e-03 -2.36866080e-02  4.05539052e-02
    3.08923210e-02 -2.93328869e-04 -5.40159075e-03 -1.00431454e-04
    2.30892042e-02 -1.06259623e-02]
 [-8.44527492e-03 -4.03476116e-03 -1.76198909e-02  3.08923210e-02
    2.45445839e-02 -2.56034242e-04 -4.03855451e-03 -7.61690728e-05
    1.72487473e-02 -7.80335729e-03]
 [-3.53377945e-04 -3.84891773e-06  1.78410386e-04 -2.93328869e-04
  -2.56034242e-04  1.45575422e-03  3.80657122e-04 -7.63355484e-06
  -8.56805820e-05 -9.42588609e-04]
 [ 2.54265382e-03  8.82100946e-04  4.39510798e-03 -5.40159075e-03
  -4.03855451e-03  3.80657122e-04  3.53852343e-03  1.70515807e-05
  -3.95756602e-03  2.56286473e-03]
 [ 2.88218529e-05  4.70211184e-05  8.68362112e-05 -1.00431454e-04
  -7.61690728e-05 -7.63355484e-06  1.70515807e-05  1.23416972e-03
  -6.20128384e-05  2.49864767e-05]
 [-8.32608753e-03 -2.47559640e-03 -1.48658217e-02  2.30892042e-02
    1.72487473e-02 -8.56805820e-05 -3.95756602e-03 -6.20128384e-05
    5.65400890e-02 -1.40376948e-02]
 [ 6.73070402e-03  1.96192447e-03  8.92339506e-03 -1.06259623e-02
  -7.80335729e-03 -9.42588609e-04  2.56286473e-03  2.49864767e-05
```

```
  -1.40376948e-02   1.99871071e-02]]
Clase 1:

 Media: fLength      0.202251
fWidth       0.112789
fSize        0.284540
fConc        0.410051
fConc1       0.316160
fAsym        0.425891
fM3Long      0.576747
fM3Trans     0.534666
fAlpha       0.488191
fDist        0.403367
dtype: float64

 Covarianza: [[ 3.10958772e-02   1.46510358e-02   1.89526334e-02 -2.63099040e-02
  -2.00211937e-02 -5.79950750e-03 -7.12038231e-03   1.81849133e-04
    2.40278783e-04   9.69916136e-03]
 [ 1.46510358e-02   1.13960053e-02   1.21535858e-02 -1.56128178e-02
  -1.19338481e-02 -2.57269228e-03 -4.04788667e-03   3.72193038e-04
    1.84462649e-03   6.38832956e-03]
 [ 1.89526334e-02   1.21535858e-02   2.05385704e-02 -2.60883372e-02
  -1.98933534e-02 -3.46753458e-03 -4.03847260e-03   1.11686643e-04
    4.77037781e-04   9.53257389e-03]
 [-2.63099040e-02 -1.56128178e-02 -2.60883372e-02   4.69194066e-02
    3.66410159e-02   4.11315763e-03   3.32824353e-03 -7.87361836e-05
  -1.53492077e-03 -9.69387344e-03]
 [-2.00211937e-02 -1.19338481e-02 -1.98933534e-02   3.66410159e-02
    3.02724543e-02   3.04986950e-03   2.48824193e-03 -7.79758226e-05
  -1.46218065e-03 -7.08262683e-03]
 [-5.79950750e-03 -2.57269228e-03 -3.46753458e-03   4.11315763e-03
    3.04986950e-03   6.36476695e-03   2.98294924e-03   1.24647710e-04
    1.13952090e-03 -2.95863097e-03]
 [-7.12038231e-03 -4.04788667e-03 -4.03847260e-03   3.32824353e-03
    2.48824193e-03   2.98294924e-03   1.55402599e-02 -2.98234261e-04
    6.55950435e-05 -2.79181056e-03]
 [ 1.81849133e-04   3.72193038e-04   1.11686643e-04 -7.87361836e-05
  -7.79758226e-05   1.24647710e-04 -2.98234261e-04   6.05469446e-03
    3.20263271e-04   2.14120281e-04]
 [ 2.40278783e-04   1.84462649e-03   4.77037781e-04 -1.53492077e-03
  -1.46218065e-03   1.13952090e-03   6.55950435e-05   3.20263271e-04
    8.28344127e-02 -4.35159779e-03]
 [ 9.69916136e-03   6.38832956e-03   9.53257389e-03 -9.69387344e-03
```

```
-7.08262683e-03 -2.95863097e-03 -2.79181056e-03  2.14120281e-04
-4.35159779e-03  2.77843307e-02]]
```

De los resultados anterior no hay mucho que analizar, solo mencionar que los valores se almacenan de manera correcta en un diccionario, el cual posee la siguiente estructura `class_gaussians = {class:{'mean': mean, 'covariance': covariance}}`, en donde la media es una serie de pandas y la covarianza es un array de numpy.

1.3.2. Implementar un clasificador Bayes con gaussianas multivariantes

En la presente subsección se realizo la implementación del clasificador Bayes utilizando Gaussianas Multivariantes.
Es importante recalcar que para realizar la implementación del clasificador solicitado, se debio programar una función capaz de manejar las Gaussianas Multivariantes, la cual se describe a continuación.

multivariate_gaussian_pdf Es la función que calcula el valor de la PDF en un punto x, multiplicando el coeficiente calculado por la exponencial del exponente calculado.
Recibe como entrada al punto x, la media de la distribución gaussiana multivariable y la matriz de covarianza de la distribución gaussiana multivariable.
Retorna lo señalado en la primera linea.

El código completo se adjunta en el siguiente bloque.

Código 18: Implementación del clasificador Bayes en base a Gaussiana Multivariante.

```python
def multivariate_gaussian_pdf(x, mean, covariance):
  n = len(mean)
  mean_arr = np.array(mean)
  diff = x - mean_arr
  coef = 1 / ((2 * np.pi) ** (n/2) * np.sqrt(np.linalg.det(covariance)))
  exponent = -0.5 * (diff @ np.linalg.inv(covariance)) @ diff.T
  return coef * np.exp(exponent)

def predict_gaussian(X, class_gaussians, threshold):
  epsilon = 1e-9
  predictions = []
  for instance in X.values:
      likelihood_ratios = []
      for cls, gaussian in class_gaussians.items():
          mean = gaussian['mean']
          covariance = gaussian['covariance']
          probability = multivariate_gaussian_pdf(instance, mean, covariance)
          likelihood_ratios.append(probability)
      probability_ratio = likelihood_ratios[1] / (likelihood_ratios[0] + epsilon)
      predicted_class = 1 if probability_ratio >= threshold else 0
```

```
    predictions.append(predicted_class)

  return predictions
```

Notar que la función **predict_gaussian** es el clasificador y en esta se utiliza un valor de épsilon muy cercano a cero, esto es utilizado para sumárselo al valor inferior de la fracción de la probabilidad **probability_ratio**, para así evitar las divisiones por cero.
La función retorna una lista llamada **predictions**, la cual contiene las clases predichas por el clasificador.

1.3.3. Curva ROC

En la presente subsección se realizara una prueba del clasificador. Para lo anterior, se utilizaran distintos umbrales (valores de Θ) y se realizara la clasificación del conjunto de prueba.
Además se obtendra la Tasa de Verdaderos Positivos y la Tasa de Falsos Positivos. Con lo anterior se generara la curva ROC para el conjunto de prueba.
El código implementado para lo anterior se adjunta en el bloque que sigue.

Código 19: Generación de la curva ROC Gaussianas Multivariantes.

```
class_gaussians = train_gaussian(X_train, y_train)
thresholds = np.concatenate([np.linspace(0, 0.0001, 3), np.linspace(0.0001, 0.001, 3), np.
    ↪ linspace(0.001, 0.01, 3), np.linspace(0.01, 0.1, 3), np.linspace(0.1, 1, 3), np.linspace(1,
    ↪  10, 3), np.linspace(10, 100, 3), np.linspace(100, 1000, 3)])
tvp_points = []
tfp_points = []
for theta in thresholds:
  predictions = predict_gaussian(X_test, class_gaussians, theta)
  tvp, tfp = calculate_TVP_and_TFP(y_test.values, predictions)
  tvp_points.append(tvp)
  tfp_points.append(tfp)
plt.figure(figsize=(8, 6))
plt.plot(tfp_points, tvp_points, marker='o', color='blue', label='ROC')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
plt.xlabel('Tasa de Falsos Positivos (TFP)')
plt.ylabel('Tasa de Verdaderos Positivos (TVP)')
plt.title('Curva ROC Multivariate Gaussian')
plt.grid(True)
plt.legend(loc='lower right')
plt.show()
```

Es importante decir que la lista de umbrales utilizadas es

[0.0e+00 5.0e-05 1.0e-04 1.0e-04 5.5e-04 1.0e-03 1.0e-03 5.5e-03 1.0e-02
 1.0e-02 5.5e-02 1.0e-01 1.0e-01 5.5e-01 1.0e+00 1.0e+00 5.5e+00 1.0e+01
 1.0e+01 5.5e+01 1.0e+02 1.0e+02 5.5e+02 1.0e+03]

A continuación se adjunta lo obtenido mediante el código anterior.

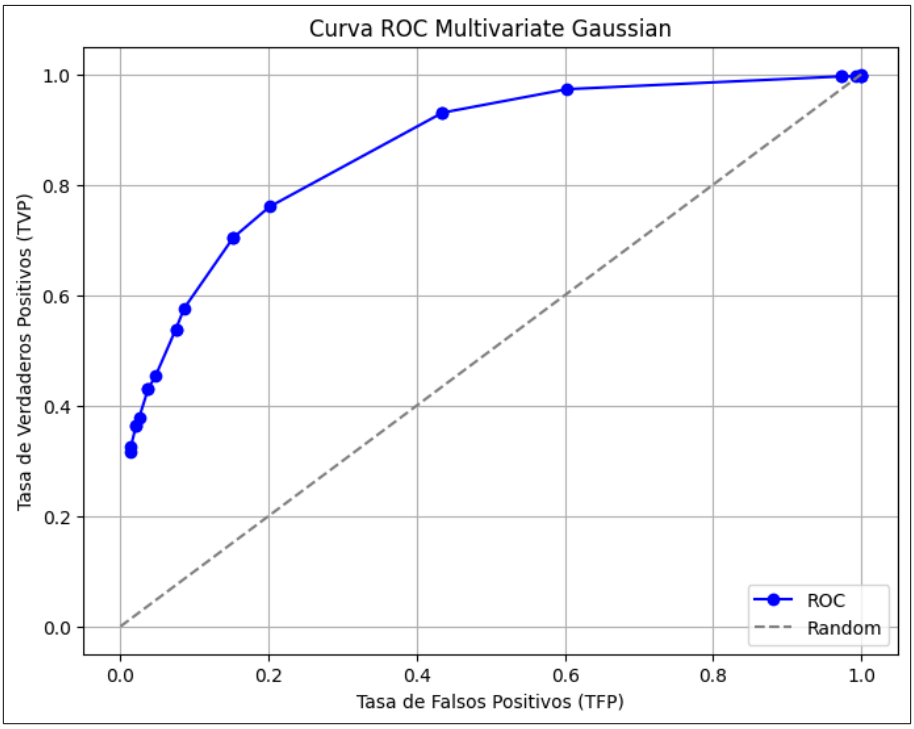


Figura 8: Curva ROC Gaussianas Multivariantes.

A continuación se realizara el análisis del gráfico 8.
Es de suma relevancia notar que el clasificador, al menos con los umbrales utilizados, nunca alcanza el punto $(0, 0)$, por lo cual se puede intuir que este no predice todas las instancias como negativas en ningún caso. Además, como el clasificador alcanza el punto $(1, 1)$ con el umbral igual a cero, por lo cual se puede intuir que este predice todas las instancias como positivas.
Además, es posible observar que en la medida que aumenta el valor del umbral, mejor comportamiento tiene el clasificador, hasta cierto punto (cercano a $(0.4, 0.9)$), desde donde comienza a decaer drásticamente el valor de **TVP**.
También, la curva generada se aleja bastante de la diagonal, que representa a un clasificador Random, por lo que es posible inferir que el clasificador implementado obtiene buenos resultados.
Por ultimo, como la curva generada esta por encima de la línea Random, se puede afirmar que este clasificador es mejor que el azar.
Es importante señalar que el análisis realizado a anteriormente para el clasificador actual presenta similitudes, por no decir que es el mismo, con el análisis realizado para 6. Lo cual implica que ambos clasificadores presentan resultados similares, aunque cabe recalcar que para umbrales distintos.
Lo anterior nos indica que ambos modelos son capaces de capturar de manera efectiva las características importantes de los datos.

1.3.4. Curva Precision-Recall

En la presente subsección se genera la curva Precision-Recall, para lo cual, se utilizaran los mismos umbrales que en la subsección previa. A partir de estos se realizara la clasificación del conjunto de prueba.
Además, se obtendrán los valores de Precision y Recall. Con lo anterior se generara la curva Precision-Recall para el conjunto de prueba.
El código implementado para lo anterior se adjunta en el bloque siguiente.

Código 20: Generación de la curva Precision-Recall Gaussianas Multivariantes.

```
class_gaussians = train_gaussian(X_train, y_train)
thresholds = np.concatenate([np.linspace(0, 0.0001, 3), np.linspace(0.0001, 0.001, 3), np.
    ↪ linspace(0.001, 0.01, 3), np.linspace(0.01, 0.1, 3), np.linspace(0.1, 1, 3), np.linspace(1,
    ↪  10, 3), np.linspace(10, 100, 3), np.linspace(100, 1000, 3)])
precision_points = []
recall_points = []
f1_scores = []
for threshold in thresholds:
  predictions = predict_gaussian(X_test, class_gaussians, threshold)
  precision, recall = calculate_precision_and_recall(y_test.values, predictions)
  precision_points.append(precision)
  recall_points.append(recall)
  f1_scores.append(2 * (precision * recall) / (precision + recall) if (precision + recall) != 0
    ↪ else 0)
best_performance_index = np.argmax(np.array(f1_scores))
best_precision = precision_points[best_performance_index]
best_recall = recall_points[best_performance_index]
best_threshold = thresholds[best_performance_index]
plt.figure(figsize=(8, 6))
plt.plot(recall_points, precision_points, marker='o', color='blue', label='Precision-Recall')
plt.plot(best_recall, best_precision, marker='o', color='orange', label=f'Mejor desempeño (
    ↪ Theta={best_threshold:.2f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Curva Precision-Recall Multivariate Gaussian')
plt.grid(True)
plt.legend()
plt.show()
```

Además de lo solicitado, se ha agregado el calculo del mejor umbral, el cual es obtenido mediante la maximización de los **f1_scores** y es señalado por el punto naranja.
A continuación se adjunta lo obtenido mediante el código anterior.

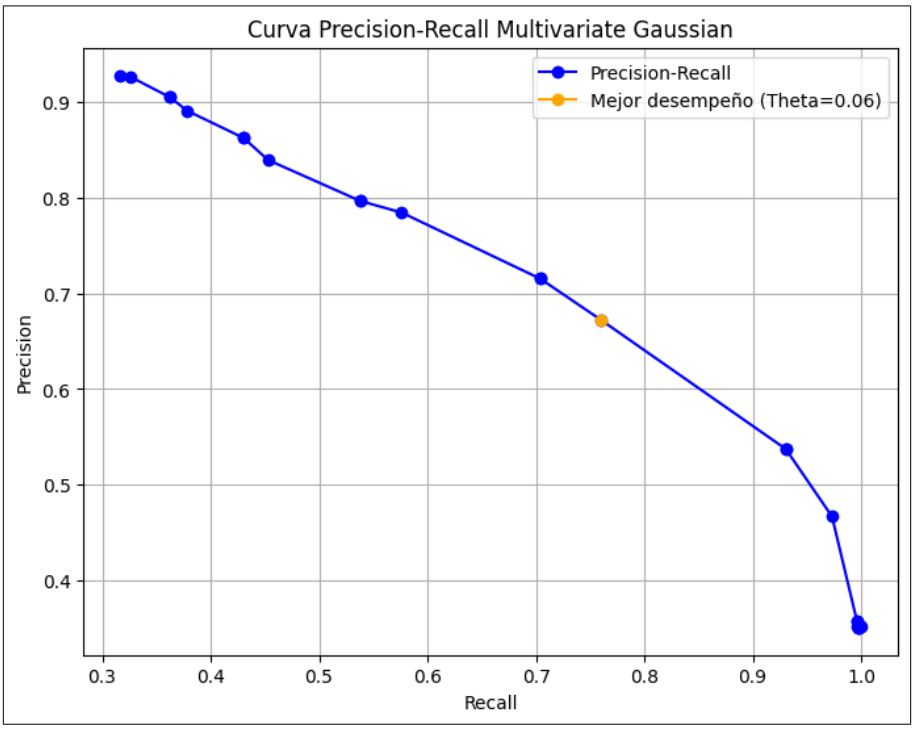


Figura 9: Curva Precision-Recall Gaussianas Multivariantes.

A continuación se realizara el análisis del gráfico 9. Es relevante notar que en el gráfico generado se puede visualizar claramente el Trade-off entre Precision y Recall, esto sucede a medida que aumenta el valor del umbral y consiste en que a medida que aumenta la Precision, el recall tiende a disminuir.
Para localizar el punto óptimo se utilizo la métrica F1-score, con lo cual se obtuvo el umbral $\Theta = 0.06$. El punto generado a partir de ese umbral es el que representa un equilibrio entre Precision y Recall.
Por ultimo, se adjuntara la lista de valores de F1-score.

[0.5204200700116686, 0.5192681977423121, 0.5192681977423121, 0.5192681977423121, 0.5208048446962298, 0.5255273013995664, 0.5255273013995664, 0.6309668039738309, 0.6812585499316005, 0.6812585499316005, 0.7134338828481234, 0.7096045197740113, 0.7096045197740113, 0.6643688065489014, 0.6422836752899197, 0.6422836752899197, 0.5883495145631068, 0.5742771684945165, 0.5742771684945165, 0.5309548793284365, 0.5176093916755603, 0.5176093916755603, 0.4828729281767956, 0.47242339832869085]

El análisis de la lista anterior se realizara a continuación. Se logra observar una tendencia, en un inicio se observa un crecimiento, hasta obtener un valor máximo 0.7134338828481234, posterior a eso los valores comienzan a decaer. Notar que si bien este es el óptimo en este intervalo, cabe la posibilidad de que exista otro superior, es decir, este es el óptimo local, no global.

1.4. Comparación de resultados

En la presente subsección se realizo la comparación de ambos clasificadores implementados. Para realizar lo anterior se compararan las curvas ROC y Precision-Recall, de ambos clasificadores, para lo cual se generaran ambas curvas en un mismo gráfico. Además se compararan los tiempos de entrenamiento de ambos clasificadores.

1.4.1. Comparativa de curvas ROC

En la presente subsección se realizara la comparación de la curva ROC de ambos clasificadores implementados.
El código implementado para la generación de las curvas ROC se adjunta en el bloque que sigue.

Código 21: Generación del gráfico comparativo de las curvas ROC.

```
class_priors, feature_bins = train_naive_bayes(X_train, y_train)
class_gaussians = train_gaussian(X_train, y_train)
thresholds = (np.concatenate((np.linspace(0, 1, 10), np.logspace(np.log10(1), np.log10(1000),
    ↪  10))), np.concatenate([np.linspace(0, 0.0001, 3), np.linspace(0.0001, 0.001, 3), np.
    ↪ linspace(0.001, 0.01, 3), np.linspace(0.01, 0.1, 3), np.linspace(0.1, 1, 3), np.linspace(1,
    ↪  10, 3), np.linspace(10, 100, 3), np.linspace(100, 1000, 3)]))
tvp_points = {'NaiveBayes': [], 'MultivariateGaussian': []}
tfp_points = {'NaiveBayes': [], 'MultivariateGaussian': []}
for threshold in thresholds[0]:
  predictions = predict_naive_bayes(X_test, class_priors, feature_bins, threshold)
  tvp, tfp = calculate_TVP_and_TFP(y_test.values, predictions)
  tvp_points['NaiveBayes'].append(tvp)
  tfp_points['NaiveBayes'].append(tfp)
for threshold in thresholds[1]:
  predictions = predict_gaussian(X_test, class_gaussians, threshold)
  tvp, tfp = calculate_TVP_and_TFP(y_test.values, predictions)
  tvp_points['MultivariateGaussian'].append(tvp)
  tfp_points['MultivariateGaussian'].append(tfp)
plt.figure(figsize=(8, 6))
plt.plot(tfp_points['NaiveBayes'], tvp_points['NaiveBayes'], marker='o', color='blue', label=
    ↪ 'Naive Bayes')
plt.plot(tfp_points['MultivariateGaussian'], tvp_points['MultivariateGaussian'], marker='o',
    ↪ color='orange', label='Multivariate Gaussian')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
plt.xlabel('Tasa de Falsos Positivos (TFP)')
plt.ylabel('Tasa de Verdaderos Positivos (TVP)')
plt.title('Curva ROC')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```

La gráfica generada se adjunta a continuación.

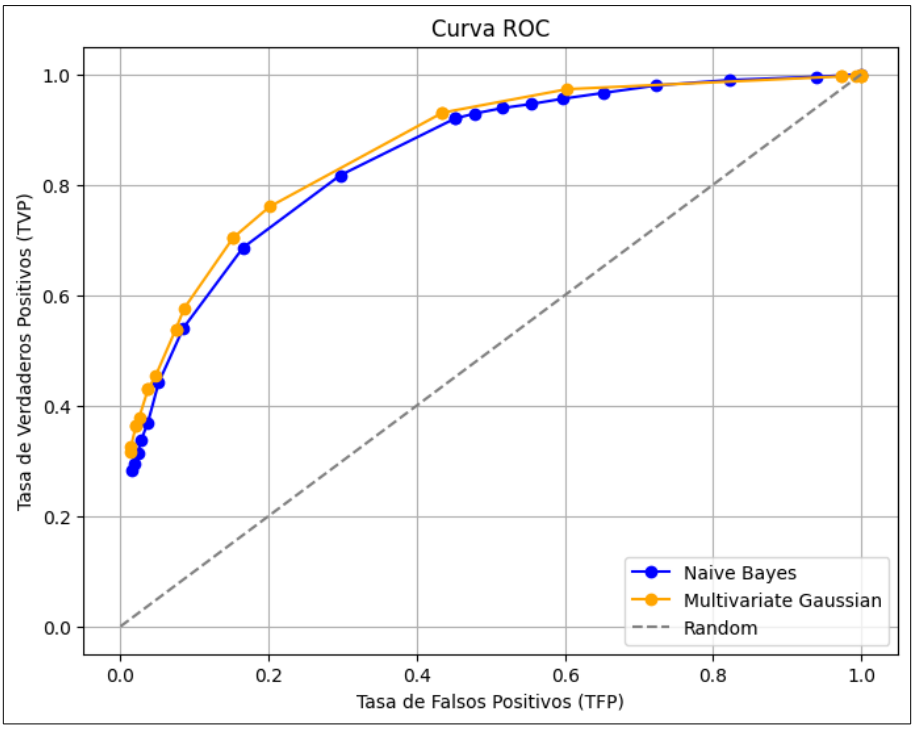


Figura 10: Gráfico comparativo de las curvas ROC de los clasificadores.

Notar que para cada clasificador se utilizó el mismo umbral que fue utilizado en subsecciones anteriores. Además, notar que la curva en azul es la correspondiente al clasificador bayesiano ingenuo, mientras que la curva en naranjo es la correspondiente al clasificador de gaussianas multivariantes.
Es claro notar que la curva naranja esta levemente por sobre la azul, lo cual implica que el clasificador de gaussianas multivariantes es levemente superior. Además, es claro notar que para el caso del clasificador con gaussianas multivariantes, los valores más pequeños tienen un comportamiento superior, esto es visible en el hecho de que tiene más puntos cercanos al $(1.0, 1.0)$.

1.4.2. Comparativa de curvas Precision-Recall

En la presente subsección se realizara la comparación de la curva Precision-Recall de ambos clasificadores implementados.
El código implementado para la generación de las curvas Precision-Recall se adjunta en el bloque que sigue.

Código 22: Generación del gráfico comparativo de las curvas Precision-Recall.

```
class_priors, feature_bins = train_naive_bayes(X_train, y_train)
class_gaussians = train_gaussian(X_train, y_train)
```

```
thresholds = (np.concatenate((np.linspace(0, 1, 10), np.logspace(np.log10(1), np.log10(1000), 10))), np.concatenate([np.linspace(0, 0.0001, 3), np.linspace(0.0001, 0.001, 3), np.linspace(0.001, 0.01, 3), np.linspace(0.01, 0.1, 3), np.linspace(0.1, 1, 3), np.linspace(1, 10, 3), np.linspace(10, 100, 3), np.linspace(100, 1000, 3)]))
precision_points = {'NaiveBayes': [], 'MultivariateGaussian': []}
recall_points = {'NaiveBayes': [], 'MultivariateGaussian': []}
for threshold in thresholds[0]:
  predictions = predict_naive_bayes(X_test, class_priors, feature_bins, threshold)
  precision, recall = calculate_precision_and_recall(y_test.values, predictions)
  precision_points['NaiveBayes'].append(precision)
  recall_points['NaiveBayes'].append(recall)
for threshold in thresholds[1]:
  predictions = predict_gaussian(X_test, class_gaussians, threshold)
  precision, recall = calculate_precision_and_recall(y_test.values, predictions)
  precision_points['MultivariateGaussian'].append(precision)
  recall_points['MultivariateGaussian'].append(recall)
plt.figure(figsize=(8, 6))
plt.plot(recall_points['NaiveBayes'], precision_points['NaiveBayes'], marker='o', color='blue', label='Naive Bayes')
plt.plot(recall_points['MultivariateGaussian'], precision_points['MultivariateGaussian'], marker='o', color='orange', label='Multivariate Gaussian')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Curva Precision-Recall')
plt.legend(loc='upper right')
plt.grid(True)
plt.show()
```

La gráfica generada se adjunta a continuación.

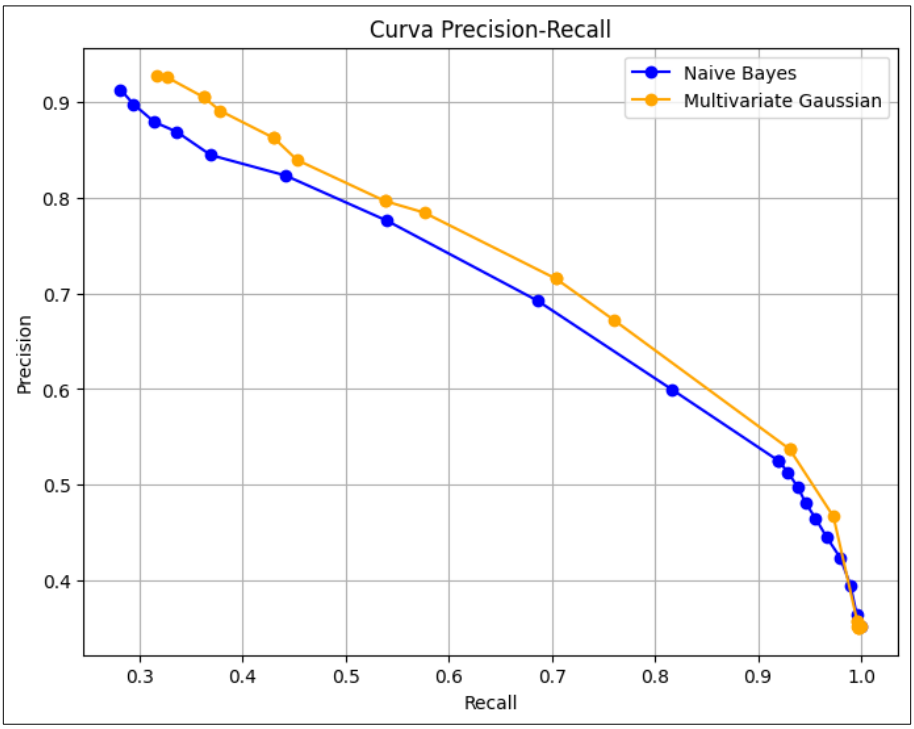


Figura 11: Gráfico comparativo de las curvas Precision-Recall de los clasificadores.

Notar que, nuevamente, para cada clasificador se utilizó el mismo umbral que fue utilizado en subsecciones anteriores. Además, notar que la curva en azul es la correspondiente al clasificador bayesiano ingenuo, mientras que la curva en naranjo es la correspondiente al clasificador de gaussianas multivariantes.
Es posible observar que las pendientes de ambas curvas son similares, donde más variación se nota es en los puntos extremos. Además, es posible notar que la curva del clasificador de gaussianas multivariantes se encuentra levemente sobre la curva del bayesiano ingenuo, por lo que es posible afirmar que el clasificador en base a gaussianas multivariantes es levemente superior al bayesiano ingenuo, pues este es capaz de mantener un valor alto de Precision mientras disminuye el valor de Recall. Aunque en el caso del clasificador bayesiano ingenuo, este alcanza un valor menor de Recall, pero sin lograr alcanzar un valor 'tan' alto de Precision, como si lo hace el otro clasificador, aunque esto puede suceder debido a los umbrales utilizados.

1.4.3. Comparativa de tiempos de entrenamiento

En la presente subsección se evaluara el tiempo requerido para el entrenamiento de ambos clasificadores. Para lo anterior, se utilizara la libreria **time**. El código implementado para la obtención del tiempo de entrenamiento de ambos clasificadores se adjunta en el bloque que sigue.

Código 23: Tiempo de entrenamiento de ambos clasificadores.

```python
# Naive Bayes Histogram
start_NB = time.time()
class_priors, feature_bins = train_naive_bayes(X_train, y_train)
end_NB = time.time()

# Multivariate Gaussian
start_MG = time.time()
class_gaussians = train_gaussian(X_train, y_train)
end_MG = time.time()

# Calculo
execution_time_NB = end_NB - start_NB
execution_time_MG = end_MG - start_MG
# Impresión de los tiempos
print(f'Tiempo de ejecución de la clasificación mediante Naive Bayes: {execution_time_NB} segundos')
print(f'Tiempo de ejecución de la clasificación mediante Multivariate Gaussian: {execution_time_MG} segundos')

# Comparar los tiempos de ejecución
if execution_time_NB < execution_time_MG:
    print('El metodo Naive Bayes fue más rapido.')
elif execution_time_NB > execution_time_MG:
    print('El metodo Multivariate Gaussian fue más rapido.')
else:
    print('Ambos metodos tardan lo mismo.')
```

La respuesta obtenida mediante la ejecución del código anterior es:

Tiempo de ejecución de la clasificación mediante Naive Bayes: 0.015002250671386719 segundos
Tiempo de ejecución de la clasificación mediante Multivariate Gaussian: 0.004461050033569336 segun
El metodo Multivariate Gaussian fue más rapido.

Se logra apreciar claramente que el clasificador en base a Gaussianas Multivariantes tiene un tiempo de entrenamiento menor.

2. Conclusiones

En la presente sección se presentaran las conclusiones generales del informe.

2.1. Aprendizajes

A continuación, se describen mis aprendizajes.

Clasificador Bayesiano He aprendido a implementar, de una manera simple y sencilla, dos tipos de clasificadores Bayesianos. Por lo que creo a que si en un futuro debiese

implementarlo nuevamente un clasificador bayesiano, sea Naive Bayes, Bayes con Gaussianas Multivariantes o algún otro, sera más sencillo para mi implementarlo.
He aprendido de primera mano que la elección de un clasificador Bayesiano particular está directamente influenciado por los histogramas de las características de las clases. Esto se ve reflejado en el caso particular de esta tarea, en donde el clasificador Bayesiano basado en Gaussianas Multivariantes tuvo un mejor rendimiento, esto debido a que casi todos los histogramas de las características tenían distribución similar a la Gaussiana.
He aprendido que es de suma relevancia estudiar las métricas de evaluación de rendimiento, pues estas nos entregan información relevante acerca de cual clasificador preferir al momento de comparar varios.

Matriz de correlación He aprendido a obtener y estudiar la matriz de correlación. Esta entrega información relevante acerca de las correlaciones lineales entre características, y se pueden agregar a las clases si se consideran como algún valor numérico.
He aprendido acerca de la visualización de datos de la matriz de correlación. En particular he aprendido que los datos más correlacionados y con un valor de correlación cercano a uno, poseen la forma de una diagonal, mientras que los datos menos correlacionados y con un valor de correlación cercano a cero, poseen una forma extraña. También aprendí que en algunos casos es bueno eliminar una característica por tener mucha o poca correlación, aunque dicha elección depende de cada problema, no es existe una regla estricta.

Conjunto de entreniamiento y prueba He aprendido a separar conjuntos de entrenamiento y prueba, y que se mantengan las proporciones entre las clases de los mismos.

Reglas de particiones He aprendio diversas reglas de calculo de particiones y que la mejor partición depende del conjunto de datos del problema, por lo que no existe una que sea superior a otra en todos los casos, por lo cual es necesario realizar pruebas.

Curvas ROC y Precision-Recall He aprendido que para obtener las curvas ROC y Precision-Recall, se requieren utilizar parámetros (umbral en el caso de Bayes) para obtener los puntos que conforman dicha curva. Además, he aprendido de primera mano que los intervalos y valores de los umbrales en el caso de un clasificador Bayesiano son de suma relevancia, pues si se utilizan valores incorrectos las curvas generadas son erradas.
He aprendido a evaluar las curvas ROC y Precision-Recall, es decir, he aprendido como discernir entre una 'buena' y una 'mala'. Además he aprendido acerca de algunas métricas para la evaluación de las mismas, las cuales no han sido implementadas, pero tienen relación con el área bajo la curva.
He aprendido a comparar curvas de distintos ROC y Precision-Recall de distintos clasificadores, logrando evaluar cual es 'mejor' a partir de las mismas.

Tiempo de entrenamiento He aprendido que esta es una metrica relevante al momento de decidir cual clasificador utilizar. En este caso particular el clasificador superior resulta ser también el que obtiene los mejores resultados en las métricas de estudio, pero podría

existir el caso en que se cumpliera solamente una cosa y ahí debiese uno ser capaz de evaluar y decidir que cosa tiene mayor peso.

2.2. Dificultades

- Uso limitado de librerías.
- Desconocimiento de **time.time()**.
- Desconocimiento a cabailidad de las metricas de evaluación de las curvas ROC y Precision-Recall.
- Realización del informe (toma mucho tiempo).
- La normalización del Data Frame, necesaria, pero no indiciada, para el caso del clasificador Bayesiano en base a las Gaussianas Multivariantes.
- Desconocimiento acerca de como seleccionar las particiones correctas.

2.3. Evaluación de resultados

Los resultados obtenidos son los que se esperan en función de la teoría, pues podemos ver que el clasificador Bayesiano en base a las Gaussianas Multivariantes es superior en este caso particular y la razón de esto es que las distribuciones de los histogramas son similares a las distribuciones gaussianas.
Además, el clasificador Bayesiano Ingenuo en base a histogramas, asume que las características son independientes entre sí, lo cual queda claro que no son, pues en la matriz de correlación, podemos ver que las primeras 5 características están bastante correlacionadas entre sí.
Por ultimo, están las métricas estudiadas en clase, las cuales son indicativas de la calidad de un modelo respecto a otro, es decir, permiten comparar los modelos. En el caso particular de esta tarea, la comparación indico que es mejor el modelo basado en las Gaussianas Multivariantes.

Anexo A. Código

Código A.1: Jupyter Notebook.

```
1 # -*- coding: utf-8 -*-
2 """Tarea 1 – Clasificador de Bayes EL4106 - Diego Vega.ipynb
3
4 Automatically generated by Colab.
5
6 Original file is located at
7    https://colab.research.google.com/drive/1gdvz4eVYvbo1-RE9XMRI1ge4j1RSw7ts
```

```
8
9  # **Importar librerias**
10 """
11
12 import pandas as pd
13 import numpy as np
14 import matplotlib.pyplot as plt
15 import seaborn as sns
16 import sklearn
17 import time
18 from sklearn.model_selection import train_test_split
19 from sklearn.preprocessing import MinMaxScaler
20
21 """# **Visualización y análisis de datos**
22
23 ## Parte 1
24 Leer el dataset magic04.data usando la función pd.read_csv(). Se debe usar los siguientes
       ↪ nombres
25 para las columnas:
26 ['fLength', 'fWidth', 'fSize', 'fConc', 'fConc1', 'fAsym', 'fM3Long',
27 'fM3Trans', 'fAlpha', 'fDist', 'class']
28 """
29
30 data = 'magic04.data'
31 column_names = ['fLength', 'fWidth', 'fSize', 'fConc', 'fConc1', 'fAsym', 'fM3Long', '
       ↪ fM3Trans', 'fAlpha', 'fDist', 'class']
32 magic04_df = pd.read_csv(data, names=column_names)
33 print("DataFrame de magic04.data con los nombres de las columnas agregados:")
34 print(magic04_df)
35
36 features = magic04_df.drop('class', axis=1)
37 scaler = MinMaxScaler()
38 normalized_features = scaler.fit_transform(features)
39 normalized_df = pd.DataFrame(normalized_features, columns=features.columns)
40 normalized_df['class'] = magic04_df['class']
41
42 print("DataFrame después de la normalización:")
43 print(normalized_df.head)
44
45 """## Parte 2
46 Separar los datos en dos subconjuntos, uno con los datos de la clase positiva (hadrón) y otro
       ↪ con los
47 de la clase negativa (no-hadrón).
48 """
49
50 hadron_df = normalized_df[normalized_df['class'] == 'h']
51 gamma_df = normalized_df[normalized_df['class'] == 'g']
```

```
print("Subconjunto de la clase hadrón:")
print(hadron_df)
print("-----------------------------------------------")
print("Subconjunto de la clase gamma primarios:")
print(gamma_df)

"""## Parte 3
Para cada característica, generar histogramas para los datos de la clase negativa y los de la clase
positiva. Para cada característica, ambos histogramas deben dibujarse usando un mismo gráfico. Para
esto, se debe usar plt.hist() con alpha=0.5. Los histogramas deben estar normalizados. Elija
un número de bins (particiones) que le parezca apropiado (investigue acerca de esto).
"""

# Particiones
bins_gamma = int(3 * np.sqrt(len(gamma_df)))
bins_hadron = int(3 * np.sqrt(len(hadron_df)))
# Histogramas
num_features = len(column_names[0:10])
for i, feature in enumerate(column_names[0:10]):
    fig, axs = plt.subplots(figsize=(8, 4))
    axs.hist(gamma_df[feature], bins=bins_gamma, density=True, alpha=0.5, color='blue', label='Clase Negativa')
    axs.hist(hadron_df[feature], bins=bins_hadron, density=True, alpha=0.5, color='orange', label='Clase Positiva')
    axs.set_xlabel('Valor')
    axs.set_ylabel('Densidad de probabilidad')
    axs.set_title(f'Histograma normalizado de: {feature}')
    axs.legend(loc='upper right')
    plt.tight_layout()
    plt.show()

"""## Parte 4
Calcular y graficar la matriz de correlación de las características, considerando el valor absoluto de las
correlaciones. Incluir en este análisis la clase, como si fuera una característica extra (usar 0 para nohadron o 1 para hadrón). Se debe usar como base la función de correlación existente en pandas. Para
graficar se debe usar la función sns.heatmap( ) de seaborn.
"""

normalized_df['class'] = normalized_df['class'].replace({'g': 0, 'h': 1})
correlation_matrix = normalized_df.corr().abs()
plt.figure(figsize=(10,8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Matriz de correlación de la data')
```

```
92 plt.show()
93
94 """## Parte 5
95 Indicar por orden cuáles son las características más correlacionadas con la clase.
96 """
97
98 class_correlations = correlation_matrix['class']
99 class_correlations_sorted = class_correlations.sort_values(ascending=False)
100 class_correlations_sorted = class_correlations_sorted.drop('class')
101 top_correlations = class_correlations_sorted
102 print('Data frame con las caracteristicas más correlacionadas con la clase:')
103 print(top_correlations)
104 print('-----------------------------------------------------------')
105 print('Las caracteristicas más correlacionadas con la clase:')
106 for i, name in enumerate(top_correlations.index.values, start=1):
107   print(f'{i}.- {name}')
108
109 """## Parte 6
110 Indicar por orden cuáles son los 3 pares de características más correlacionadas entre sí.
      ↪ Averigüe
111 sobre unstack() y sort_values().
112 """
113
114 mask = pd.DataFrame(False, index=correlation_matrix.index, columns=correlation_matrix.
      ↪ columns)
115 for i in range(len(correlation_matrix.columns)):
116   for j in range(i+1, len(correlation_matrix.columns)):
117     value = correlation_matrix.iloc[i, j]
118     if value not in correlation_matrix.values[:i, j]:
119       mask.iloc[i, j] = True
120 correlation_matrix_unique_values = correlation_matrix.mask(~mask)
121 correlation_pairs_sorted = correlation_matrix_unique_values.unstack().sort_values(
      ↪ ascending=False).dropna()
122 top_correlation_pairs = correlation_pairs_sorted[correlation_pairs_sorted !=1].head(3)
123 print('Data frame con las primeras tres caracteristicas más correlacionadas entre sí:')
124 print(top_correlation_pairs)
125 print('-----------------------------------------------------------')
126 print('Primeras tres caracteristicas más correlacionadas entre sí:')
127 for i, (pair, correlation) in enumerate(top_correlation_pairs.items(), start=1):
128   feature1, feature2 = pair
129   print(f'{i}.- {feature1} y {feature2}, con una correlación de {correlation}')
130
131 """## Parte 7
132 Graficar las dos características más correlacionadas en un scatter plot (gráfico de dispersión)
133 """
134
135 feature1, feature2 = top_correlation_pairs.index[0]
```

```
feature1_data = normalized_df[feature1]
feature2_data = normalized_df[feature2]

plt.figure(figsize=(8,6))
plt.scatter(feature1_data, feature2_data, alpha=0.5)
plt.xlabel(feature1)
plt.ylabel(feature2)
plt.title(f'Gráfico de dispersión de {feature1} vs {feature2}')
plt.grid(True)
plt.show()

"""## Parte 8
Indicar por orden cuáles son los 3 pares de características menos correlacionadas entre sí
"""

last_correlation_pairs = correlation_pairs_sorted[(correlation_pairs_sorted !=1)].
    ↪ sort_values(ascending=True).head(3)
print('Data frame con las ultimas tres caracteristicas más correlacionadas entre sí:')
print(last_correlation_pairs)
print('------------------------------------------------------------')
print('Ultimas tres caracteristicas más correlacionadas entre sí:')
for i, (pair, correlation) in enumerate(last_correlation_pairs.items(), start=1):
  feature1, feature2 = pair
  print(f'{i}.- {feature1} y {feature2}, con una correlación de {correlation}')

"""## Parte 9
Graficar las dos características menos correlacionadas en un scatter plot. Compare el gráfico
    ↪ obtenido
con el del punto 7.
"""

feature1, feature2 = last_correlation_pairs.index[0]
feature1_data = normalized_df[feature1]
feature2_data = normalized_df[feature2]

plt.figure(figsize=(8,6))
plt.scatter(feature1_data, feature2_data, alpha=0.5)
plt.xlabel(feature1)
plt.ylabel(feature2)
plt.title(f'Gráfico de dispersión de {feature1} vs {feature2}')
plt.grid(True)
plt.show()

"""Comparar graficos:

# Clasificación usando naive Bayes e histogramas

```

```
Se pide programar, entrenar y calibrar un clasificador naive Bayes para determinar en forma probabilística
si un candidato corresponde a la clase positiva (hadrón) o a la negativa (no-hadrón). La regla de decisión
del mencionado clasificador está dada por:
$$
\frac{P(\textit{características|positiva})}{P(\textit{caracrísticas|negativa})} \geq \Theta
$$
El umbral $\Theta$ depende de los costos asociados a cada decisión ($c_{\textit{positiva}} $ y $c_{\textit{negativa}}$) y de las probabilidades a priori:
$$
\Theta=\frac{c_{\text{positiva}}P(\text{negativa})}{c_{\text{negativa}}P(\text{positiva})}
$$
En este caso concreto, las distribuciones de probabilidad condicional, i.e. las verosimilitudes, de ambos conjuntos no se tienen, por lo que deberá estimarlas utilizando histogramas normalizados.

## Parte 10
Dividir la base de datos en 2 conjuntos representativos: entrenamiento (80 %) y prueba (20 %).
Compruebe la representatividad de estos, verificando si la proporción de ambas clases en cada
conjunto se mantiene cercana a la proporción del conjunto completo. Se recomienda usar la función
train_test_split() para hacer la división, usando un random_state fijo.
"""

X = normalized_df.drop('class', axis=1)
y = normalized_df['class']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=3)
original_proportion = [round(len(gamma_df)/len(magic04_df), 4), round(len(hadron_df)/len(magic04_df), 4)]
train_proportion = [round((y_train==0).sum()/len(y_train), 4), round((y_train==1).sum()/len(y_train), 4)]
test_proportion = [round((y_test==0).sum()/len(y_test), 4), round((y_test==1).sum()/len(y_test), 4)]
print(original_proportion, train_proportion, test_proportion)

"""## Parte 11
Implemente un clasificador naive Bayes que use histogramas para representar las verosimilitudes.
"""

bins = int(3 * np.sqrt(len(X_test)))

```

```
214 def train_naive_bayes(X, y, bins=bins):
215   classes = [0, 1]
216   assert len(classes) == 2
217   class_priors = {}
218   feature_bins = {}
219   for cls in classes:
220       cls_samples = X[y == cls]
221       class_priors[cls] = len(cls_samples)/len(y)
222       feature_bins[cls] = []
223       for col in X.columns:
224           hist, bin_edges = np.histogram(cls_samples[col], bins=bins, density=True)
225           hist_normalized = hist / sum(hist)
226           feature_bins[cls].append((hist_normalized, bin_edges))
227 
228   return class_priors, feature_bins
229 
230 def find_bin_index(bin_edges, value):
231   for i in range(len(bin_edges) - 1):
232     if bin_edges[i] <= value < bin_edges[i + 1]:
233       return i
234   return -1
235 
236 def calculate_likelihood(instance, feature_bins, epsilon=1e-9):
237     likelihood = 1
238     for index, hist_and_edges in enumerate(feature_bins):
239         hist, bin_edges = hist_and_edges
240         bin_index = find_bin_index(bin_edges, instance[index])
241         if bin_index >= 0 and bin_index < len(hist) - 1:
242             left_bin = hist[bin_index]
243             right_bin = hist[bin_index + 1]
244             bin_width = bin_edges[1] - bin_edges[0]
245             interpolation_factor = (instance[index] - bin_edges[bin_index]) / bin_width
246             left_bin_smoothed = left_bin + epsilon
247             right_bin_smoothed = right_bin + epsilon
248             likelihood *= left_bin_smoothed + (right_bin_smoothed - left_bin_smoothed) *
    ↪ interpolation_factor
249         else:
250             likelihood *= epsilon
251 
252     return likelihood
253 
254 
255 def predict_naive_bayes(X, class_priors, feature_bins, theta):
256     predictions = []
257     for __, instance in X.iterrows():
258       likelihood_0 = calculate_likelihood(instance, feature_bins[0])
259       likelihood_1 = calculate_likelihood(instance, feature_bins[1])
```

```
260       prob_ratio = (class_priors[1] * likelihood_1) / (class_priors[0] * likelihood_0)
261       predicted_class = 1 if prob_ratio >= theta else 0
262       predictions.append(predicted_class)
263
264     return predictions
265
266 def calculate_theta(c_pos, p_pos, c_neg, p_neg):
267   return (c_pos * p_neg) / (c_neg * p_pos)
268
269 """## Parte 12
270 Utilice la aproximación Naive Bayes y calcule los histogramas de cada clase a partir de las
      ↪ muestras
271 del conjunto de entrenamiento. Elija un número de bins que le parezca apropiado, y use el
      ↪ mismo
272 número de bins para todas las características. Se recomienda normalizar los histogramas.
273 """
274
275 def calculate_class_histograms(X, y, bins=bins):
276   classes = [0, 1]
277   class_histograms = {}
278   for cls in classes:
279     cls_samples = X[y == cls]
280     class_histograms[cls] = []
281     for col in X.columns:
282       hist, bin_edges = np.histogram(cls_samples[col], bins=bins, density=True)
283       hist_normalized = hist / sum(hist)
284       class_histograms[cls].append((hist_normalized, bin_edges))
285
286   return class_histograms
287
288 class_histograms = calculate_class_histograms(X_train, y_train)
289
290 """## Parte 13
291 Grafique los histogramas usando plt.bar(), con alpha = 0.5. En cada gráfico se debe
      ↪ superponer
292 el histograma de una característica para la clase positiva y el histograma para la clase
      ↪ negativa.
293 """
294
295 classes = [0, 1]
296 num_features = len(X_train.columns)
297 num_classes = len(classes)
298 for i, col in enumerate(X_train.columns):
299     hist0, bin_edges0 = class_histograms[classes[0]][i]
300     hist1, bin_edges1 = class_histograms[classes[1]][i]
301     bar_width = (bin_edges0[1] - bin_edges0[0]) * 0.4
302     fig, axs = plt.subplots(figsize=(8, 4))
```

```
    axs.bar(bin_edges0[:-1], hist0, width=bar_width, alpha=0.5, label=f'Clase {classes[0]}')
    axs.bar(bin_edges1[:-1], hist1, width=bar_width, alpha=0.5, label=f'Clase {classes[1]}')
    axs.set_xlabel('Valor')
    axs.set_ylabel('Densidad de probabilidad')
    axs.set_title(f'Histogramas de {col} para las clases {classes[0]} y {classes[1]}')
    axs.legend()
    plt.tight_layout()
    plt.show()

"""## Parte 14
Probar el clasificador: utilizando distintos valores de Θ, clasificar el conjunto de prueba y calcular la
Tasa de Verdaderos Positivos y Tasa de Falsos Positivos, para luego generar la curva ROC ( TVP vs
TFP) para el conjunto de prueba. Recuerde que Θ puede tomar valores entre 0 e , pero no todo el
rango entrega información importante.
"""

def confusion_matrix(y_true, y_predict):
  matrix = [[0, 0], [0, 0]]
  for true, pred in zip(y_true, y_predict):
    matrix[true][pred] += 1

  return matrix[0][0], matrix[0][1], matrix[1][0], matrix[1][1]

def calculate_TVP_and_TFP(y, y_pred):
  tn, fp, fn, tp = confusion_matrix(y, y_pred)
  TVP = tp / (tp + fn) if (tp + fn) != 0 else 0
  TFP = fp / (fp + tn) if (fp + tn) != 0 else 0

  return TVP, TFP

class_priors, feature_bins = train_naive_bayes(X_train, y_train)
thresholds = np.concatenate((np.linspace(0, 1, 10), np.logspace(np.log10(1), np.log10(1000), 10)))
tvp_points = []
tfp_points = []
predictions_list = []
for theta in thresholds:
  predictions = predict_naive_bayes(X_test, class_priors, feature_bins, theta)
  tvp, tfp = calculate_TVP_and_TFP(y_test.values, predictions)
  tvp_points.append(tvp)
  tfp_points.append(tfp)
plt.figure(figsize=(8, 6))
plt.plot(tfp_points, tvp_points, marker='o', color='blue', label='ROC')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
```

```
346 plt.xlabel('Tasa de Falsos Positivos (TFP)')
347 plt.ylabel('Tasa de Verdaderos Positivos (TVP)')
348 plt.title('Curva ROC Naive Bayes')
349 plt.legend(loc='lower right')
350 plt.grid(True)
351 plt.show()
352
353 """## Parte 15
354 Genere la curva precisión-recall para el conjunto de prueba.
355 """
356
357 def calculate_precision_and_recall(y, y_pred):
358   tn, fp, fn, tp = confusion_matrix(y, y_pred)
359   precision = tp / (tp + fp) if tp + fp != 0 else 0
360   recall = tp / (tp + fn) if tp + fn != 0 else 0
361   return precision, recall
362
363 class_priors, feature_bins = train_naive_bayes(X_train, y_train)
364 thresholds = np.concatenate((np.linspace(0, 1, 10), np.logspace(np.log10(1), np.log10(1000),
      ↪ 10)))
365 precision_points = []
366 recall_points = []
367 f1_scores = []
368 for theta in thresholds:
369   predictions = predict_naive_bayes(X_test, class_priors, feature_bins, theta)
370   precision, recall = calculate_precision_and_recall(y_test.values, predictions)
371   precision_points.append(precision)
372   recall_points.append(recall)
373   f1_scores.append(2 * (precision * recall) / (precision + recall) if (precision + recall) != 0
      ↪ else 0)
374 best_performance_index = np.argmax(np.array(f1_scores))
375 best_precision = precision_points[best_performance_index]
376 best_recall = recall_points[best_performance_index]
377 best_threshold = thresholds[best_performance_index]
378 plt.figure(figsize=(8, 6))
379 plt.plot(recall_points, precision_points, marker='o', color='blue', label='Precision-Recall')
380 plt.plot(best_recall, best_precision, marker='o', color='orange', label=f'Mejor desempeño (
      ↪ Theta={best_threshold:.2f})')
381 plt.xlabel('Recall')
382 plt.ylabel('Precision')
383 plt.title('Curva Precision-Recall Naive Bayes')
384 plt.grid(True)
385 plt.legend()
386 plt.show()
387
388 """# Clasificación usando gaussianas multivariantes
```

```
En esta sección se pide programar un clasificador basado en gaussianas multivariantes, el cual no usa la
aproximación naive Bayes.

## Parte 16
Entrenar un modelo gaussiano multidimensional para cada clase, calculando la media y covarianza, a
partir del conjunto de entrenamiento. Puede basarse en las funciones np.mean( ) y np.cov( ) de
numpy.
"""

def train_gaussian(X, y):
  classes = [0, 1]
  class_gaussians = {}
  for cls in classes:
    cls_samples = X[y == cls]
    mean = np.mean(cls_samples, axis=0)
    covariance = np.cov(cls_samples, rowvar=False)
    class_gaussians[cls] = {'mean': mean, 'covariance': covariance}

  return class_gaussians

class_gaussians = train_gaussian(X_train, y_train)
for cls, gaussian in class_gaussians.items():
  print(f'Clase {cls}:')
  print(f'\n Media: {gaussian["mean"]}')
  print(f'\n Covarianza: {gaussian["covariance"]}')

"""## Parte 17
Implemente el clasificador
"""

def multivariate_gaussian_pdf(x, mean, covariance):
  n = len(mean)
  mean_arr = np.array(mean)
  diff = x - mean_arr
  coef = 1 / ((2 * np.pi) ** (n/2) * np.sqrt(np.linalg.det(covariance)))
  exponent = -0.5 * (diff @ np.linalg.inv(covariance)) @ diff.T
  return coef * np.exp(exponent)

def predict_gaussian(X, class_gaussians, threshold):
  epsilon = 1e-9
  predictions = []
  for instance in X.values:
      likelihood_ratios = []
      for cls, gaussian in class_gaussians.items():
```

```
433         mean = gaussian['mean']
434         covariance = gaussian['covariance']
435         probability = multivariate_gaussian_pdf(instance, mean, covariance)
436         likelihood_ratios.append(probability)
437     probability_ratio = likelihood_ratios[1] / (likelihood_ratios[0] + epsilon)
438     predicted_class = 1 if probability_ratio >= threshold else 0
439     predictions.append(predicted_class)
440
441  return predictions
442
443 """## Parte 18
444 Probar el clasificador: utilizando distintos valores de Θ, clasificar el conjunto de prueba y
    ↪ calcular la
445 Tasa de Verdaderos Positivos y Tasa de Falsos Positivos para luego generar la curva ROC (
    ↪ TVP vs
446 TFP) para el conjunto de prueba.
447 """
448
449 class_gaussians = train_gaussian(X_train, y_train)
450 thresholds = np.concatenate([np.linspace(0, 0.0001, 3), np.linspace(0.0001, 0.001, 3), np.
    ↪ linspace(0.001, 0.01, 3), np.linspace(0.01, 0.1, 3), np.linspace(0.1, 1, 3), np.linspace(1,
    ↪  10, 3), np.linspace(10, 100, 3), np.linspace(100, 1000, 3)])
451 tvp_points = []
452 tfp_points = []
453 for theta in thresholds:
454  predictions = predict_gaussian(X_test, class_gaussians, theta)
455  tvp, tfp = calculate_TVP_and_TFP(y_test.values, predictions)
456  tvp_points.append(tvp)
457  tfp_points.append(tfp)
458 plt.figure(figsize=(8, 6))
459 plt.plot(tfp_points, tvp_points, marker='o', color='blue', label='ROC')
460 plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
461 plt.xlabel('Tasa de Falsos Positivos (TFP)')
462 plt.ylabel('Tasa de Verdaderos Positivos (TVP)')
463 plt.title('Curva ROC Multivariate Gaussian')
464 plt.grid(True)
465 plt.legend(loc='lower right')
466 plt.show()
467
468 """## Parte 19
469 Genere la curva precisión-recall para el conjunto de prueba.
470 """
471
472 class_gaussians = train_gaussian(X_train, y_train)
473 thresholds = np.concatenate([np.linspace(0, 0.0001, 3), np.linspace(0.0001, 0.001, 3), np.
    ↪ linspace(0.001, 0.01, 3), np.linspace(0.01, 0.1, 3), np.linspace(0.1, 1, 3), np.linspace(1,
    ↪  10, 3), np.linspace(10, 100, 3), np.linspace(100, 1000, 3)])
```

```
precision_points = []
recall_points = []
f1_scores = []
for threshold in thresholds:
  predictions = predict_gaussian(X_test, class_gaussians, threshold)
  precision, recall = calculate_precision_and_recall(y_test.values, predictions)
  precision_points.append(precision)
  recall_points.append(recall)
  f1_scores.append(2 * (precision * recall) / (precision + recall) if (precision + recall) != 0
    ↪ else 0)
best_performance_index = np.argmax(np.array(f1_scores))
best_precision = precision_points[best_performance_index]
best_recall = recall_points[best_performance_index]
best_threshold = thresholds[best_performance_index]
plt.figure(figsize=(8, 6))
plt.plot(recall_points, precision_points, marker='o', color='blue', label='Precision-Recall')
plt.plot(best_recall, best_precision, marker='o', color='orange', label=f'Mejor desempeño (
    ↪ Theta={best_threshold:.2f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Curva Precision-Recall Multivariate Gaussian')
plt.grid(True)
plt.legend()
plt.show()

"""# Comparación de resultados

## Parte 20
Grafique las curvas ROC de los clasificadores entrenados en un único gráfico. Analice las mé
    ↪ tricas
obtenidas por ambos clasificadores.
"""

class_priors, feature_bins = train_naive_bayes(X_train, y_train)
class_gaussians = train_gaussian(X_train, y_train)
thresholds = (np.concatenate((np.linspace(0, 1, 10), np.logspace(np.log10(1), np.log10(1000),
    ↪  10))), np.concatenate([np.linspace(0, 0.0001, 3), np.linspace(0.0001, 0.001, 3), np.
    ↪ linspace(0.001, 0.01, 3), np.linspace(0.01, 0.1, 3), np.linspace(0.1, 1, 3), np.linspace(1,
    ↪  10, 3), np.linspace(10, 100, 3), np.linspace(100, 1000, 3)]))
tvp_points = {'NaiveBayes': [], 'MultivariateGaussian': []}
tfp_points = {'NaiveBayes': [], 'MultivariateGaussian': []}
for threshold in thresholds[0]:
  predictions = predict_naive_bayes(X_test, class_priors, feature_bins, threshold)
  tvp, tfp = calculate_TVP_and_TFP(y_test.values, predictions)
  tvp_points['NaiveBayes'].append(tvp)
  tfp_points['NaiveBayes'].append(tfp)
for threshold in thresholds[1]:
```

```
  predictions = predict_gaussian(X_test, class_gaussians, threshold)
  tvp, tfp = calculate_TVP_and_TFP(y_test.values, predictions)
  tvp_points['MultivariateGaussian'].append(tvp)
  tfp_points['MultivariateGaussian'].append(tfp)
plt.figure(figsize=(8, 6))
plt.plot(tfp_points['NaiveBayes'], tvp_points['NaiveBayes'], marker='o', color='blue', label=
    ↪ 'Naive Bayes')
plt.plot(tfp_points['MultivariateGaussian'], tvp_points['MultivariateGaussian'], marker='o',
    ↪ color='orange', label='Multivariate Gaussian')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
plt.xlabel('Tasa de Falsos Positivos (TFP)')
plt.ylabel('Tasa de Verdaderos Positivos (TVP)')
plt.title('Curva ROC')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

"""## Parte 21
Grafique las curvas precisión-recall de los clasificadores entrenados en un único gráfico.
    ↪ Analice las
métricas obtenidas por ambos clasificadores.
"""

class_priors, feature_bins = train_naive_bayes(X_train, y_train)
class_gaussians = train_gaussian(X_train, y_train)
thresholds = (np.concatenate((np.linspace(0, 1, 10), np.logspace(np.log10(1), np.log10(1000),
    ↪  10))), np.concatenate([np.linspace(0, 0.0001, 3), np.linspace(0.0001, 0.001, 3), np.
    ↪ linspace(0.001, 0.01, 3), np.linspace(0.01, 0.1, 3), np.linspace(0.1, 1, 3), np.linspace(1,
    ↪  10, 3), np.linspace(10, 100, 3), np.linspace(100, 1000, 3)]))
precision_points = {'NaiveBayes': [], 'MultivariateGaussian': []}
recall_points = {'NaiveBayes': [], 'MultivariateGaussian': []}
for threshold in thresholds[0]:
  predictions = predict_naive_bayes(X_test, class_priors, feature_bins, threshold)
  precision, recall = calculate_precision_and_recall(y_test.values, predictions)
  precision_points['NaiveBayes'].append(precision)
  recall_points['NaiveBayes'].append(recall)
for threshold in thresholds[1]:
  predictions = predict_gaussian(X_test, class_gaussians, threshold)
  precision, recall = calculate_precision_and_recall(y_test.values, predictions)
  precision_points['MultivariateGaussian'].append(precision)
  recall_points['MultivariateGaussian'].append(recall)
plt.figure(figsize=(8, 6))
plt.plot(recall_points['NaiveBayes'], precision_points['NaiveBayes'], marker='o', color='blue'
    ↪ , label='Naive Bayes')
plt.plot(recall_points['MultivariateGaussian'], precision_points['MultivariateGaussian'],
    ↪ marker='o', color='orange', label='Multivariate Gaussian')
plt.xlabel('Recall')
```

```
554 plt.ylabel('Precision')
555 plt.title('Curva Precision-Recall')
556 plt.legend(loc='upper right')
557 plt.grid(True)
558 plt.show()
559
560 """## Parte 22
561 Compare los tiempos requeridos para entrenar ambos clasificadores.
562 """
563
564 # Naive Bayes Histogram
565 start_NB = time.time()
566 class_priors, feature_bins = train_naive_bayes(X_train, y_train)
567 end_NB = time.time()
568
569 # Multivariate Gaussian
570 start_MG = time.time()
571 class_gaussians = train_gaussian(X_train, y_train)
572 end_MG = time.time()
573
574 # Calculo
575 execution_time_NB = end_NB - start_NB
576 execution_time_MG = end_MG - start_MG
577 # Impresión de los tiempos
578 print(f'Tiempo de ejecución de la clasificación mediante Naive Bayes: {execution_time_NB}
        ↪ segundos')
579 print(f'Tiempo de ejecución de la clasificación mediante Multivariate Gaussian: {
        ↪ execution_time_MG} segundos')
580
581 # Comparar los tiempos de ejecución
582 if execution_time_NB < execution_time_MG:
583     print('El metodo Naive Bayes fue más rapido.')
584 elif execution_time_NB > execution_time_MG:
585     print('El metodo Multivariate Gaussian fue más rapido.')
586 else:
587     print('Ambos metodos tardan lo mismo.')
```